NEUROREPL.JL: A MODEL CONTEXT PROTOCOL SERVER FOR INTERACTIVE JULIA DEVELOPMENT [DRAFT - NOT FOR DISTRIBUTION]

j underscore u at julia discourse

Introduction **6**

1 Technical Overview **7**

1.1 NeuroREPL vs Julia REPL 7

1.2 Architecture Overview 7

1.3 Dependencies 8

1.4 Environment Structure 8

1.5 Implementation 8

1.6 Limitations 8

1.7 Potential Enhancements 8

2 Server Configuration & Settings **9**

2.1 Network Configuration [`SERVER_IP[] SERVER_PORT[]`] 9

2.2 Starting the Server [`start_server!() send_sse_keepalive()`] . . 9

2.3 Stopping the server [`stop_server!()`] 10

2.4 Server Behavior Settings [`verbose_server!() show_repl!() agent_capture!()`] 10

2.5 Performance Characteristics 10

2.6 Server Functions And Shortcuts Reference 11

3 Dual MCP Transport **12**

3.1 Supported Transports 12

3.1.1 SSE Transport (MCP 2024-11-05) 12

3.1.2 Streamable HTTP (MCP 2025-03-26+) 12

3.1.3 Health Check 12

3.2 Transport Architecture 12

3.2.1 Core Functions 12

3.2.2 Request Routing Workflow 13

3.3 Request Flow Diagrams 13

3.3.1 General Request Flow 13

3.3.2 SSE Transport Flow 13

3.3.3 HTTP Transport Flow 14

3.3.4 Health Check Flow 14

3.4 Connection and Session Management 15

3.5 Transport Statistics [`server_status()`] 15

4 Worker Process Isolation **16**

4.1 Worker Initialization and Control [`init_worker!() interrupt_worker!() worker_status!()`] 16

4.2 State Management [`restart_worker()` `reset_worker_env()`] . . . 16

4.3 Worker Execution Function . . . 17

4.4 Output Capture [`agent_capture!()`] . . . 17

4.5 Usage Patterns . . . 18

4.5.1 Starting the Server . . . 18

4.5.2 Agent Workflow Example . . . 18

4.5.3 Parallel Processing Workflow . . . 19

4.5.4 User Interactive REPL Modes [`mode_status()` `@worker`] . . . 20

5 Agent Tools Overview . . . 22

5.1 Core Execution (1 tool) . . . 22

5.2 Package Management (3 tools) . . . 22

5.3 Code Introspection (9 tools) . . . 22

5.4 File Operations (3 tools) . . . 22

5.5 Performance Analysis (2 tools) . . . 22

5.6 Code Analysis (4 tools) . . . 22

5.7 Testing (1 tool) . . . 23

5.8 Server Control (5 tools) . . . 23

6 Testing & Validation . . . 24

6.1 Tested MCP Clients . . . 24

6.2 Known Issues . . . 25

7 Contributing & Development . . . 26

7.1 Project Structure . . . 26

7.2 Development Workflow . . . 26

7.3 Adding New Tools . . . 26

8 License & Attribution . . . 27

9 Conclusion . . . 28

References . . . 29

Appendix A: NeuroREPL v. 0.2 - Tool Reference Guide . . . 30

Tool Categories Summary . . . 30

Core Execution . . . 30

execute_julia . . . 30

Package Management . . . 31

pkg_add . . . 31

pkg_rm . . . 32

pkg_status . . . 32
Code Introspection . . . 32
list_variables . . . 32
typeinfo . . . 33
methodswith . . . 33
methods . . . 34
names . . . 34
help . . . 35
which . . . 35
varinfo . . . 35
apropos . . . 36
File Operations . . . 36
include . . . 36
pwd . . . 37
cd . . . 37
Performance Analysis . . . 38
time . . . 38
profile . . . 38
Code Analysis . . . 39
macroexpand . . . 39
code_lowered . . . 39
code_typed . . . 39
code_warntype . . . 40
Testing . . . 40
test . . . 40
Server Control . . . 41
server_status . . . 41
worker_status . . . 41
interrupt_worker . . . 42
restart_worker . . . 42
reset_worker_env . . . 43
Appendix B: NeuroREPL v. 0.2 - Screenshots . . . 44
NeuroREPL Starting Sequence - Part 1/2 . . . 45
NeuroREPL Starting Sequence - Part 2/2 . . . 46
Appendix C: NeuroREPL v. 0.1 (Legacy Version) - Prompt Examples . 47
What packages are installed in the current NeuroREPL environment? . 48

What tools are available for Julia NeuroREPL MCP - please check directly in Julia? . . . 49

What transport protocol does NeuroREPL use? [It is the MCP (Model Context Protocol) server that provides the Julia-related tools you have access to.] . . . 50

Is the NeuroREPL server protected by any network-security measures, or does it run on an unsecured localhost port? . . . 51

Let's test NeuroREPL. Please add 2 + 2 in Julia. . . . 52

Plot a heatmap (8 x 8) in NeuroREPL via MCP with UnicodePlots.jl and show me the result. Remember to check if the UnicodePlots.jl is installed in the current environment. . . . 53

Appendix D: NeuroREPL v. 0.2 - Limitations & Potential Enhancements (UNEDITED) . . . 54

1. No LSP Integration . . . 54
2. No Debugger Integration . . . 55
3. Single Worker Process for MCP Requests . . . 56
4. No Security Layer . . . 57
5. Limited Error Recovery . . . 57
6. No Binary/Image Output Support . . . 59
7. No Explicit Session Management in MCP Protocol . . . 61
8. No Full REPL Features in Worker Mode . . . 62
9. Multi-Model Solution Validation and Ranking: . . . 64
10. Parallel Agent Collaboration (Anthropic-style): . . . 67
11. Meta-AI System: Competitive + Collaborative: . . . 71

Introduction

`NeuroREPL.jl` is a `Julia` implementation of the Model Context Protocol (MCP) that transforms the `Julia REPL` into a programmatic API for AI agents. `NeuroREPL` provides process-isolated code execution, dual MCP transport support, and introspection tools specifically designed for agent workflows.

The system is built on `Malt.jl` for worker process isolation, `HTTP.jl` for dual transport (`SSE` and `Streamable HTTP`), and provides 28 specialized tools across 8 functional categories including core execution, package management, code introspection, performance analysis, and server control. The architecture separates the MCP server from the execution environment, ensuring that code execution cannot crash the server while maintaining persistent state across tool calls.

`NeuroREPL.jl` is inspired by the MIT-licensed `MCPRepl.jl` project but represents a significant rewrite focused on improved agent communication, and enhanced debugging capabilities. The implementation is contained in a single ~2000 line Julia module for ease of modification and understanding.

Testing has been performed with Kiro CLI v1.25.0 (AWS), Crush v0.43.0 (Charm), and OpenCode v1.2.10. While core functionality is solid, the package remains in active development and is not production ready.

This document provides technical details on architecture, transport protocols, worker isolation, tool capabilities, and usage patterns.

NOTE: This document was prepared with some assistance from Large Language Models (LLMs).

1 Technical Overview

1.1 NeuroREPL vs Julia REPL

- Julia REPL: Interactive shell for humans
- NeuroREPL: Programmatic API for agents
- Difference: NeuroREPL adds MCP protocol, tools, isolation
- Limitations: When in :w (worker) mode, there are no completions, no access to REPL modes with shortcuts, etc.

Key Innovation:

- Process-isolated code execution using Malt.jl with persistent state,
- Dual MCP transport support (SSE + Streamable HTTP)
- Julia introspection tools designed specifically for AI agent workflows
- Logging
- Debugging
- Statistics

1.2 Architecture Overview

```
┌──────────────────────────────────────────────────────────────────────────┐
│                          AI Agent Layer                                  │
│                         (any MCP client)                                 │
└──────────────────────────┬──────────────────────────────────────────────┘
                           │
                           │ JSON-RPC 2.0 Messages
                           │
┌──────────────────────────▼──────────────────────────────────────────────┐
│                       MCP Transport Layer                                │
│  ┌──────────────────────────┬──────────────────────────────────┐        │
│  │   SSE Transport          │   Streamable HTTP                │        │
│  │   (MCP 2024-11-05)       │   (MCP 2025-03-26+)              │        │
│  │  ┌────────────────────────┼────────────────────────────────┐        │
│  │  │ GET /sse               │ POST /message                  │        │
│  │  │ POST /sse/message      │ Direct request/response        │        │
│  │  │ Streaming responses    │ Session management             │        │
│  │  └────────────────────────┴────────────────────────────────┘        │
│  │                                                                      │
│  │                    GET / (Health Check)                              │
└──────────────────────────┬──────────────────────────────────────────────┘
                           │
┌──────────────────────────▼──────────────────────────────────────────────┐
│                       JSON-RPC 2.0 Handler                               │
│  ┌──────────────────────────────────────────────────────────────────┐   │
│  │  Methods: initialize, tools/list, tools/call                     │   │
│  │  Request routing, error handling, response formatting            │   │
│  └──────────────────────────────────────────────────────────────────┘   │
└──────────────────────────┬──────────────────────────────────────────────┘
                           │
┌──────────────────────────▼──────────────────────────────────────────────┐
│                         Tool Execution Layer                             │
│  ┌──────────────┬──────────────┬──────────────┬──────────────┐          │
│  │   Core       │   Package    │ Introspection│ Performance  │          │
│  │  Execution   │ Management   │    Tools     │   Analysis   │          │
│  └──────────────┴──────────────┴──────────────┴──────────────┘          │
└──────────────┬──────────────┬──────────────┬──────────────┬──────────────┘
               ▼              ▼              ▼              ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                       Malt.jl Worker Process                             │
│  ┌──────────────────────────────────────────────────────────────────┐   │
│  │  Isolated Julia REPL Environment                                 │   │
│  │  • Persistent state across tool calls                            │   │
│  │  • IOCapture for stdout/stderr                                   │   │
│  │  │  • Clean environment (no inherited ENV/packages)              │   │
│  │  • __neurorepl_eval__() execution function                       │   │
│  │  • Separate Project.toml in worker_env/                          │   │
│  └──────────────────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────────────────┘
```

1.3 Dependencies

```
# Server Dependencies
HTTP        # Web server and HTTP client library
JSON        # JSON parsing and serialization
REPL        # Julia's Read-Eval-Print Loop utilities
Malt        # Lightweight worker process management
IOCapture   # Capture stdout/stderr output
Dates       # Date and time functionality
```

```
# Worker Dependencies
IOCapture   # Capture stdout/stderr output
```

This separation ensures:

- Server dependencies don't pollute agent workspace
- Agent can install any packages without affecting server

1.4 Environment Structure

```
NeuroREPL_Distributed_Feb26/
├── Project.toml              # Server dependencies
├── Manifest.toml             # Server lockfile
├── src/
│   └── NeuroREPL.jl          # Main module (server + tools)
├── worker_env/               # Worker environment (isolated)
│   ├── Project.toml          # Worker dependencies
│   └── Manifest.toml         # Worker lockfile
├── TOOLS.md                  # Tool documentation
├── ReadMe.md                 # User guide
└── nr                        # Bash launch script
    └── nr.jl                 # Julia launch script
```

1.5 Implementation

`NeuroREPL.jl` is implemented as a single file Julia module (~2000 lines) with clearly delineated sections for transport handling, tool definitions, and worker management. The server uses `HTTP.jl` for the web layer, `JSON.jl` for protocol serialization, and `Malt.jl` for isolated worker processes. All MCP protocol logic, tool handlers, and REPL shortcuts are contained in `src/NeuroREPL.jl`, making the codebase easy to modify. The modular structure separates concerns: **i)** transport layer handles SSE and HTTP connections, **ii)** protocol handler processes JSON-RPC messages, **iii)** tool layer executes operations, and **iv)** the worker process provides isolated Julia execution. This architecture enables the server to handle multiple transport protocols simultaneously while isolating network handling from code execution and at the same time supporting isolation of server operations from agent working environment.

1.6 Limitations

- No LSP Integration
- No Debugger Integration
- No Full REPL Features in Worker Mode
- No Binary/Image Output Support
- Limited Error Recovery
- Single Worker Process for MCP Requests
- No Explicit Session Management via MCP Protocol
- No Security Layer

1.7 Potential Enhancements

- Parallel Agent Collaboration
- Multi-Model Solution Validation and Ranking
- Meta-AI System: Competitive + Collaborative Multi-Model Approach

2 Server Configuration & Settings

2.1 Network Configuration [`SERVER_IP[]` `SERVER_PORT[]`]

Default configuration: 127.0.0.1:3001.

Common configurations:

Use Case	IP	Port	Notes
Local development	127.0.0.1	3001	Only accessible from same machine
Network access	0.0.0.0	3001	Accessible from other machines
Specific interface	100.01.02.03	3001	Only on that network interface
Avoid conflicts	Any	8080	Use different port if 3001 is taken

```
# Change settings (requires server restart)
SERVER_IP[] = "127.0.0.1"  # localhost only (secure)
SERVER_IP[] = "0.0.0.0"    # all interfaces (open)
SERVER_IP[] = "100.01.02.03"  # specific interface

SERVER_PORT[] = 3001  # default
SERVER_PORT[] = 8080  # alternative port

# Apply changes
stop_server!()
start_server!()
```

2.2 Starting the Server [`start_server!()` `send_sse_keepalive()`]

From Julia REPL:

```
using NeuroREPL

# Start with defaults (port 3001, IP from SERVER_IP[])
start_server!()

# Start with custom port
start_server!(8080)

# Start with custom port and IP
start_server!(8080, "127.0.0.1")  # localhost only
start_server!(3001, "0.0.0.0")    # listen on all interfaces
```

From interactive prompt (REPL shortcut):

```
julia> :start!<Enter>
Enter port (or press Enter for 3001): 8080
Enter IP (or press Enter for 100.01.02.03): 127.0.0.1
✓ NeuroREPL Server started on http://127.0.0.1:8080
```

From shell:

```
# Using launch script (uses defaults)
./nr

# Or directly with Julia
julia -e 'using NeuroREPL; start_server!()'
```

NOTE: SSE Keepalive Configuration

Keepalive prevents connection timeouts but may cause issues with some clients. Disabled by default for maximum compatibility. Enable if enocuntering server (transport) configuration issues.

```
# NOTE: Disable keepalive
# (default, for Crush by Charm compatibility)

send_sse_keepalive(false)


# Enable keepalive with 5s interval
send_sse_keepalive(true, 5.0)

# Or use REPL shortcut
julia> :ka<Enter>
```

2.3 Stopping the server [`stop_server!()`]

```
stop_server!()  # Stops server, keeps worker running
# Or use shortcut: :stop!
```

2.4 Server Behavior Settings [`verbose_server!()` `show_repl!()` `agent_capture!()`]

```
# Verbose HTTP logging (default: true)
verbose_server!(true)   # Show all HTTP requests/responses
verbose_server!(false)  # Silent operation
# Shortcuts: :vt (verbose true), :vf (verbose false)
```

```
# Show code in REPL (default: true)
show_repl!(true)   # Echo "julia> code" before result
show_repl!(false)  # Only show result
# Shortcuts: :at (agent true), :af (agent false)
```

```
# Capture output (default: true)
agent_capture!(true)   # Capture println/warnings (slight overhead)
agent_capture!(false)  # No capture (faster, output lost)
# Shortcuts: :at (agent true), :af (agent false)
```

```
# Quick toggle from REPL Example:
julia> :vf<Enter>  # Disable verbose logging
julia> :rf<Enter>  # Hide code echo
julia> :af<Enter>  # Disable output capture
```

2.5 Performance Characteristics

Startup Times

- Server startup: ~2-3s (load `HTTP.jl`, `JSON.jl`, `Malt.jl`)
- Worker startup: ~0.5-1s (`Malt.jl` worker spawn)
- First tool call: ~0.5-1s (worker initialization)
- Subsequent calls: <10ms (worker already running)

Execution Overhead

- IOCapture overhead: ~1-5ms per call (when enabled)
- JSON-RPC overhead: ~1-2ms per request
- Network overhead: <1ms (localhost)

2.6 Server Functions And Shortcuts Reference

Reference:

Function	Shortcut	Description
MODE CONTROL		
-	:w	Switch to worker mode
-	:s	Switch to server mode
mode_status()	:ms	Check current mode
STATUS & MONITORING		
server_status()	:ss	Check server status
worker_status!()	:ws	Check worker status
WORKER MANAGEMENT		
init_worker!()	:iw	Initialize worker
interrupt_worker!()	:i!	Interrupt long-running code
restart_worker!()	:r!	Restart worker (keeps packages)
reset_worker_env!()	:re!	Reset environment (removes packages)
SERVER CONTROL		
start_server!()	:start!	Start server (prompts for port/IP)
stop_server!()	:stop!	Stop server
send_sse_keepalive()	:ka	Configure SSE keepalive
CONFIGURATION TOGGLES		
verbose_server!(true/false)	:vt / :vf	Toggle verbose logging
show_repl!(true/false)	:rt / :rf	Toggle REPL output display
agent_capture!(true/false)	:at / :af	Toggle agent output capture
HELP & INFO		
-	:hs	Show shortcuts help
-	:hat	Show agent tools help
-	:hnr	Show full NeuroREPL help
-	:q!	Exit Julia

```
# Example usage of shortcuts:
julia> :ss<Enter>      # Check server status
julia> :ws<Enter>      # Check worker status
julia> :vt<Enter>      # Enable verbose mode
julia> :r!<Enter>      # Restart worker
```

3 Dual MCP Transport

NeuroREPL implements SSE and Streamable HTTP MCP transport protocols simultaneously:

3.1 Supported Transports

3.1.1 SSE Transport (MCP 2024-11-05)

- Endpoints: `GET /sse` (establish stream) `POST /sse/message` (send requests)
- Protocol: Legacy MCP protocol (backwards compatible, removed from the latest MCP specification)
- Connection: Stateful (long-lived SSE stream)
- State tracking: `CURRENT_SSE[]` stores active `SSEConnection` object
- Response delivery: Asynchronous (queued to channel, returns 202 Accepted, sent via SSE events)

3.1.2 Streamable HTTP (MCP 2025-03-26+)

- Endpoint: `POST /message` (all JSON-RPC requests)
- Protocol: Modern MCP protocol (recommended)
- Connection: Stateless (per-request HTTP)
- State tracking: `SESSIONS` dict with session IDs for protocol state
- Response delivery: Synchronous (direct HTTP response, returns 200 OK or 204 No Content)
- Features: Session management, simpler architecture

3.1.3 Health Check

- Endpoint: `GET /`
- Returns: Server status, transport information, and statistics
- Use case: Monitoring, debugging, connection verification

Comparison:

Aspect	SSE Transport	HTTP Transport
MCP Version	2024-11-05 (legacy)	2025-03-26+ (modern)
Connection model	Stateful	Stateless
Connection lifetime	Long-lived	Per-request
State storage	SSEConnection	SESSIONS dict
Response delivery	Async (queued)	Synchronous
Response code	202 Accepted	200 OK / 204 No Content
Keepalive needed	Yes (optional)	No
MCP protocol state	Per-connection	Per-session
Recommended for	Legacy clients	Modern clients

Both transports share the same tool execution layer and worker process.

3.2 Transport Architecture

3.2.1 Core Functions

1. **`create_handler(tools)`** - Main request router
   - Creates HTTP stream handler function
   - Routes requests to appropriate transport handler
   - Handles health checks and unknown endpoints
   - Returns: HTTP.Stream handler function
2. **`handle_jsonrpc(request, tools_dict, session_id, transport)`** - JSON-RPC protocol handler
   - Processes MCP protocol methods (`initialize`, `tools/list`, `tools/call`, etc.)
   - Manages sessions and connection metadata
   - Executes tool handlers and formats responses
   - Transport-agnostic (works for both SSE and HTTP)
   - Returns: JSON-RPC response dict or `nothing` for notifications

3.2.2 Request Routing Workflow

```
HTTP.Stream (incoming request)
    |
    ├─→ GET /sse
    |   └─→ SSE Connection Handler
    |       ├─ Create SSEConnection object
    |       ├─ Send endpoint event
    |       ├─ Loop: read from channel, send SSE events
    |       └─ Cleanup on disconnect
    |
    ├─→ POST /sse/message
    |   └─→ SSE Message Handler
    |       ├─ Parse JSON-RPC request
    |       ├─ Call handle_jsonrpc()
    |       ├─ Queue response to SSEConnection.channel
    |       └─ Return 202 Accepted
    |
    ├─→ POST /message
    |   └─→ HTTP Message Handler
    |       ├─ Parse JSON-RPC request
    |       ├─ Extract session ID from headers
    |       ├─ Call handle_jsonrpc()
    |       └─ Return direct HTTP response (200/204)
    |
    ├─→ GET /
    |   └─→ Health Check Handler
    |       └─ Return server status JSON
    |
    └─→ Other
        └─→ Return 404 Not Found
```

3.3 Request Flow Diagrams

3.3.1 General Request Flow

```
Client     Transport    Protocol      Tool         Worker
           Layer        Handler       Layer        Process
  |           |             |             |             |
  ├─ HTTP ──>|             |             |             |
  |  Request  |             |             |             |
  |           ├─ Route ───>|             |             |
  |           |  Endpoint   |             |             |
  |           |             ├─ Parse ──>|             |
  |           |             |  JSON      |             |
  |           |             |             ├─ Lookup ──>|
  |           |             |             |  Tool       |
  |           |             |             |             ├─ Execute
  |           |             |             |             |  Code
  |           |             |             |<─ Return ──┤
  |           |             |             |  Result     |
  |           |             |<─ Format ──┤             |
  |           |             |  JSON      |             |
  |           |<─ Send ────┤             |             |
  |           |  Response   |             |             |
  |<─ HTTP ──┤             |             |             |
  |  Response |             |             |             |
```

3.3.2 SSE Transport Flow

```
Client                                    Server (NeuroREPL)
  |                                              |
  ├─ GET /sse ──────────────────────────────────>|
  |                                              ├─ Create SSEConnection
  |                                              ├─ Assign Connection ID
  |                                              ├─ Set CURRENT_SSE[]
```

```
  |                                          ├─ Open Channel{String}(32)
  |<─ event: endpoint ──────────────────────┤
  |   data: http://SERVER_IP:PORT/sse/message  |
  |                                          |
  |                                          ├─ Loop: read from channel
  |                                          |   send SSE events
  |                                          |
  ├─ POST /sse/message ──────────────────────>|
  |   {"jsonrpc":"2.0","method":"tools/call"...} |
  |                                          ├─ Parse JSON-RPC
  |                                          ├─ Call handle_jsonrpc()
  |                                          ├─ Execute tool
  |                                          ├─ Queue response
  |<─ 202 Accepted ─────────────────────────┤
  |                                          |
  |<─ event: message ───────────────────────┤
  |   data: {"jsonrpc":"2.0","result":{...}}  |
  |                                          |
  |<─ : ping ───────────────────────────────┤ (keepalive)
  |                                          |
  ├─ Close Connection ───────────────────────>|
  |                                          ├─ Set active = false
  |                                          ├─ Close channel
  |                                       ├─ Set CURRENT_SSE[] = nothing
  |                                          ├─ Exit loop
  |                                          |
```

3.3.3 HTTP Transport Flow

```
Client                                  Server (NeuroREPL)
  |                                          |
  ├─ POST /message ──────────────────────────>|
  |   Header: Mcp-Session-Id: <session-id>   |
  |   {"jsonrpc":"2.0","method":"initialize"...} |
  |                                       ├─ Increment HTTP_CONNECTION_COUNT
  |                                          ├─ Extract session ID
  |                                          ├─ Parse JSON-RPC
  |                                          ├─ Call handle_jsonrpc()
  |                                          ├─ Store session in SESSIONS
  |<─ 200 OK ───────────────────────────────┤
  |   Header: Mcp-Session-Id: <session-id>   |
  |   {"jsonrpc":"2.0","result":{...}}       |
  |                                          |
  ├─ POST /message ──────────────────────────>|
  |   Header: Mcp-Session-Id: <session-id>   |
  |   {"jsonrpc":"2.0","method":"tools/call"...} |
  |                                          ├─ Extract session ID
  |                                          ├─ Parse JSON-RPC
  |                                          ├─ Call handle_jsonrpc()
  |                                          ├─ Execute tool on worker
  |<─ 200 OK ───────────────────────────────┤
  |   {"jsonrpc":"2.0","result":{...}}       |
  |                                          |
  ├─ POST /message ──────────────────────────>|
  |   {"jsonrpc":"2.0","method":"initialized"}  | (notification)
  |                                          ├─ Parse JSON-RPC
  |                                          ├─ Call handle_jsonrpc()
  |                                          ├─ Return nothing
  |<─ 204 No Content ───────────────────────┤
  |                                          |
```

3.3.4 Health Check Flow

```
Client                                  Server (NeuroREPL)
  |                                          |
  ├─ GET / ──────────────────────────────────>|
  |                                          ├─ Check CURRENT_SSE[] status
  |                                          ├─ Build status JSON
  |<─ 200 OK ───────────────────────────────┤
  |   {"status":"ok","transport":"dual", |
  |    "sse_active":true/false}              |
  |                                          |
```

3.4 Connection and Session Management

SSE Connection Management

```
# SSE connection state
mutable struct SSEConnection
    id::Int                     # Unique connection ID
    channel::Channel{String}    # Message queue for this connection
    active::Bool                # Connection status
    client::String              # Client name/version
    connected_at::Float64       # Connection timestamp
    protocol::String            # MCP protocol version
    capabilities::String        # Client capabilities
    request_count::Int          # Requests on this connection
end

# Global state
const CURRENT_SSE = Ref{Union{Nothing,SSEConnection}}(nothing)
const SSE_CONNECTION_COUNT = Ref{Int}(0)  # Total SSE connections
```

HTTP Session Management

```
# Session tracking (session IDs provided by client in headers)
const SESSIONS = Dict{String,Dict{String,Any}}()

# Session stored on initialize
# (if session ID provided in Mcp-Session-Id header)
if session_id !== nothing
    SESSIONS[session_id] = Dict(
        "initialized" => true,
        "created" => time()
    )
end
```

```
# Connection metadata tracking

# Incremented on initialize
const HTTP_CONNECTION_COUNT = Ref{Int}(0)

# Last client name/version
const HTTP_LAST_CLIENT = Ref{String}("")

# Last protocol version
const HTTP_PROTOCOL = Ref{String}("")

# Last client capabilities
const HTTP_CAPABILITIES = Ref{String}("")

# Last connection timestamp
const HTTP_CONNECTED_AT = Ref{Float64}(0.0)
```

3.5 Transport Statistics [`server_status()`]

Accessible via `server_status()` tool or function.

```
const TOTAL_REQUESTS = Ref{Int}(0)     # All requests (SSE + HTTP)
const SSE_REQUESTS = Ref{Int}(0)       # SSE transport requests
const HTTP_REQUESTS = Ref{Int}(0)      # HTTP transport requests
const LAST_REQUEST_TIME = Ref{Float64}(0.0) # Time of last request
```

Note: Connection counts (`SSE_CONNECTION_COUNT`, `HTTP_CONNECTION_COUNT`) are tracked in their respective management sections above.

4 Worker Process Isolation

`NeuroREPL` uses `Malt.jl` (the same library powering `Pluto.jl` notebooks) for worker process management. `Malt` workers are completely isolated (separate process, no shared memory) and can be interrupted without killing them (native `Malt.interrupt()` implemented as `interrupt_worker!()` in `NeuroREPL`). The workers start with clean `Julia` session (no precompiled packages from parent) and add only `IOCapture` dependency.

Architecture Benefits:

- Code execution cannot crash the MCP server
- Worker can be restarted without restarting server
- Clean environment prevents package conflicts
- Fast startup enables quick session resets

4.1 Worker Initialization and Control [`init_worker!()` `interrupt_worker!()` `worker_status!()`]

Worker Initialization:

- `init_worker!()` - Spawns `Malt` worker process (automatic on first `execute_julia` call)
- Worker activates isolated environment in `worker_env/` directory
- Installs `IOCapture.jl` for output capture
- Defines `__neurorepl_eval__()` function for code execution
- Typically takes ~0.5-1s on first call, subsequent calls reuse existing worker

Worker Control and State Inspection:

- `interrupt_worker!()` - Send interrupt signal to stop long-running code (cross-platform, including Windows)
- `worker_status!()` - Check if worker is running, dead, or not initialized
- Uses `Malt.interrupt()` and `Malt.isrunning()` for process management

Why `Malt.jl` over `Distributed.jl`?

Feature	Malt.jl	Distributed.jl
Startup time	0.5-1s	2-3s
Environment isolation	Clean (no inheritance)	Inherits parent ENV
Interrupt support	Cross-platform (Windows!)	Unix-only
Design purpose	Process sandboxing	Distributed computing
Zombie processes	Rare	More common

4.2 State Management [`restart_worker()` `reset_worker_env()`]

The `Malt.jl` worker process maintains a persistent `Julia` REPL session. State persists across tool calls until explicitly reset:

```
# Call 1: Define variables
execute_julia(code: "x = 10; y = 20")

# Call 2: Variables remain available
execute_julia(code: "z = x + y")  # Returns: 30

# Call 3: Load package
execute_julia(code: "using DataFrames")

# Call 4: Package remains loaded
execute_julia(code: "df = DataFrame(a=1:3)")
```

State reset options:

- `restart_worker()` = Restarts worker, keeps packages in worker_env: fresh workspace, same packages available
- `reset_worker_env()` = Deletes worker_env, completely fresh start: fresh workspace + fresh package environment

Tool	Effect	Use Case
restart_worker()	• Restart worker process • Clear all workspace state • Keep installed packages	• Clean slate for new task • Resolve state-related bugs
reset_worker_env()	• Restart worker process • Delete worker_env directory • Remove all installed packages	• Resolve package conflicts • Start with completely fresh environment • Remove all dependencies

4.3 Worker Execution Function

The core execution happens in `__neurorepl_eval__()` defined in the worker. This function is called by the server via `Malt.remote_eval_fetch()` when executing code:

```
function __neurorepl_eval__(code_str::String; capture::Bool=true)
    # Parse code into expression
    ex = Meta.parse("begin\n" * code_str * "\nend")

    result = nothing
    captured_out = ""
    exception = nothing

    if capture
        # Use IOCapture to capture stdout/stderr
        c = IOCapture.capture() do
            try
                result = eval(ex)
            catch e
                exception = e
            end
        end
        captured_out = c.output
    else
        # Direct execution (faster, no capture)
        try
            result = eval(ex)
        catch e
            exception = e
        end
    end

    return (result=result, output=captured_out
                  , error="", exception=exception)
end
```

4.4 Output Capture [`agent_capture!()`]

`NeuroREPL` uses `IOCapture.jl` to capture stdout/stderr and optionally pass it to the agent:

```
# With CAPTURE_OUTPUT[] = true (default)
execute_julia(code: """
    println("Computing...")
    result = 2 + 2
    println("Result: \$result")
    result
""")
# Returns: "Computing...\nResult: 4\n4"

# With CAPTURE_OUTPUT[] = false
# Returns: "4" (println output goes to worker stdout, not captured)
```

Configuration:

- `agent_capture!(true)` - Captures and passes to agent (default, slight overhead)
- `agent_capture!(false)` - No capture, output goes to worker stdout (faster, output lost)

Known Limitations:

1. Separately stored stdout/stderr objects
   - Capture doesn't work if code prints to a previously stored `stdout`/`stderr` reference:

```
const original_stdout = stdout
execute_julia(code: """
    println("captured")                       # Captured
    println(original_stdout, "not captured")  # Not captured
""")
```

2. Stream closure after capture
   - `stdout`/`stderr` objects obtained during capture become invalid after the capture ends:

```
execute_julia(code: "captured_stdout = stdout")

# Error: stream closed
execute_julia(code: "println(captured_stdout, \"test\")")
```

   - This is because IOCapture uses temporary redirect streams that are cleaned up after each call.
3. ANSI color sequences (Julia ≤1.5)
   - On Julia 1.5 and earlier, captured output may not contain ANSI color escape sequences
   - Julia 1.6+ properly preserves colors in captured output

Workaround: Use standard `println()` and `@show` macros for output - these work correctly with IOCapture.

4.5 Usage Patterns

4.5.1 Starting the Server

```
# From Julia REPL
using NeuroREPL
start_server!()

# Or from shell
./nr  # Launches Julia and starts server
```

4.5.2 Agent Workflow Example

```
# 1. Check environment
status()

# 2. Install needed packages
pkg_add(packages: ["DataFrames", "CSV"])

# 3. Load packages and define functions
execute_julia(code: """
    using DataFrames, CSV

    function process_data(path)
        df = CSV.read(path, DataFrame)
        return describe(df)
    end
""")
```

```
# 4. Execute analysis
execute_julia(code: "process_data(\"data.csv\")")

# 5. Check performance
time(expr: "process_data(\"data.csv\")")

# 6. Profile if slow
profile(expr: "process_data(\"data.csv\")")

# 7. Check type stability
code_warntype(call: "process_data(\"data.csv\")")
```

4.5.3 Parallel Processing Workflow

Agents can use `Distributed.jl` inside the worker for parallel computation:

```
# 1. Install Distributed (part of Julia standard library)
execute_julia(code: """
    using Distributed

    # Add 4 worker processes
    addprocs(4)

    println("Workers: ", nworkers())
""")
```

```
# 2. Define function on all workers
execute_julia(code: """
    @everywhere function parallel_work(x)
        # Simulate computation
        sleep(0.1)
        return x^2
    end
""")
```

```
# 3. Execute parallel computation
execute_julia(code: """
    # Parallel map
    result = pmap(parallel_work, 1:100)
    println("Processed ", length(result), " items in parallel")
    result[1:5]  # Show first 5 results
""")
```

```
# 4. Use @distributed for reductions
execute_julia(code: """
    total = @distributed (+) for i in 1:1000
        i^2
    end
    println("Sum of squares: ", total)
    total
""")
```

```
# 5. Clean up workers when done
execute_julia(code: """
    rmprocs(workers())
    println("Workers removed")
""")
```

Note: Distributed workers run inside the Malt worker process. This provides:

- Parallel computation within a single MCP session
- Shared state across parallel tasks (via `@everywhere`)
- Standard Julia parallel programming patterns

However, this doesn't enable concurrent MCP requests - those are still serialized to the single Malt worker.

4.5.4 User Interactive REPL Modes [`mode_status()` `@worker`]

NeuroREPL provides dual-mode REPL with multiple ways to execute code on the worker:

Check Current Mode:

```
julia> mode_status()
"Server mode"  # or "Worker mode"

# Or use REPL shortcut
julia> :ms<Enter>
```

Mode 1: Worker Mode (`:w`)

Switch to worker mode for interactive execution:

```
# Server mode (default)
julia> :ss  # Check server status (runs on server)

# Switch to worker mode
julia> :w<Enter>

# Worker mode - all code runs on worker
julia> x = 10
10

julia> y = 20
20

julia> x + y
30

# Switch back to server
julia> :s<Enter>
```

Mode 2: `@worker` Macro from Server Mode

Execute worker code without switching modes:

```
# Stay in server mode, execute single expression on worker
julia> @worker x = 1 + 1
2

julia> @worker println("Hello from worker")
Hello from worker

julia> @worker using DataFrames
```

Mode 3: `@worker` Block from Server Mode

Execute multi-line code blocks on worker:

```
# Server mode - execute block on worker
julia> @worker begin
           x = 1
           y = 2
           z = x + y
           println("Result: $z")
           z
       end
Result: 3
3
```

```
# More complex example
julia> @worker begin

           using Statistics
           data = rand(100)
           μ = mean(data)
           σ = std(data)
           println("Mean: $μ, Std: $σ")
           (mean=μ, std=σ)
       end
Mean: 0.523, Std: 0.289
(mean = 0.523, std = 0.289)
```

Comparison:

Method	Mode Switch	Use Case	Example
:w mode	Yes	Interactive exploration	:w x = 10
@worker expr	No	Quick single command	@worker x = 10
@worker begin	No	Multi-line scripts	@worker begin x = 10 y = 20 end

All three methods:

- Execute in the same worker process
- Share the same state (variables, loaded packages)
- Capture output (if `agent_capture!(true)`)
- Display in REPL (if `show_repl!(true)`)

Example showing shared state:

```
# Define variable in :w mode
julia> :w
julia> x = 42
42
julia> :s

# Access from @worker macro
julia> @worker x + 8
50

# Modify in @worker block
julia> @worker begin
           x = x * 2
           println("x is now: $x")
       end
x is now: 84

# Verify in :w mode
julia> :w
julia> x
84

# Worker mode
julia> x = 10  # Runs on worker via @worker macro
10

# Switch back to server
julia> :s<Enter>
```

5 Agent Tools Overview

`NeuroREPL.jl` provides **28 tools** organized into **8 functional categories**:

```
Tool Distribution by Category
```

Category	Count	Percentage	Bar Chart
Core Execution	1	3.6%	▓
Package Management	3	10.7%	▓▓▓
Code Introspection	9	32.1%	▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓
File Operations	3	10.7%	▓▓▓
Performance Analysis	2	7.1%	▓▓
Code Analysis	4	14.3%	▓▓▓▓
Testing	1	3.6%	▓
Server Control	5	17.9%	▓▓▓▓▓
Total	28	100.0%	

5.1 Core Execution (1 tool)

- `execute_julia` - Execute arbitrary Julia code with optional REPL display

5.2 Package Management (3 tools)

- `pkg_add` - Install packages to worker environment
- `pkg_rm` - Remove packages from worker environment
- `pkg_status` - Show installed packages and versions

5.3 Code Introspection (9 tools)

- `list_variables` - List all variables in Main module workspace
- `typeinfo` - Analyze type hierarchy, fields, and properties
- `methodswith` - Find methods using a specific type
- `methods` - List all methods of a function
- `names` - List exported/all names in a module
- `help` - Get documentation for functions/types
- `which` - Show method dispatch for specific call
- `varinfo` - Show variable names, types, and memory usage
- `apropos` - Search documentation by keyword

5.4 File Operations (3 tools)

- `include` - Load and execute Julia source files
- `pwd` - Get current working directory
- `cd` - Change working directory

5.5 Performance Analysis (2 tools)

- `time` - Measure execution time and allocations
- `profile` - Profile code to find bottlenecks

5.6 Code Analysis (4 tools)

- `macroexpand` - Expand macros to see generated code
- `code_lowered` - Show lowered (desugared) IR
- `code_typed` - Show type-inferred IR
- `code_warntype` - Analyze type stability with warnings

5.7 Testing (1 tool)

- `test` - Run package test suites

5.8 Server Control (5 tools)

- `server_status` - Show server status, transport info, statistics
- `worker_status` - Check worker process health
- `interrupt_worker` - Interrupt long-running code
- `restart_worker` - Restart worker process for clean state
- `reset_environment` - Delete worker_env and restart fresh

6 Testing & Validation

6.1 Tested MCP Clients

- Kiro CLI v. 1.25.0 (AWS) - Streamable HTTP transport

```
# Streamable HTTP Config: .kiro/settings/mcp.json

{
  "mcpServers": {
    "NeuroREPL": {
      "url": "http://100.01.02.03:3001/message",
      "transport": "http"
    }
  }
}

# NOTE: Is SSE supported?
```

- Crush v. 0.43.0 (Charm) - SSE transport

```
# SSE Config: Project Working Directory/crush.json
# Run Crush with: crush --cwd .

{
  "$schema": "https://charm.land/crush.json",
  "options": {
    "disable_metrics": true
  },
  "mcp": {
    "NeuroREPL": {
      "type": "sse",
      "url": "http://100.01.02.03:3001/sse",
      "timeout": 120
    }
  }
}
```

```
# Streamable HTTP Config: Project Working Directory/crush.json
# Run Crush with: crush --cwd .

{
  "$schema": "https://charm.land/crush.json",
  "options": {
    "disable_metrics": true
  },
  "mcp": {
    "NeuroREPL": {
      "type": "http",
      "url": "http://100.01.02.03:3001/message",
      "timeout": 120,
      "disabled": false
    }
  }
}
```

- OpenCode v. 1.2.10 - Streamable HTTP transport

```
# STreamable HTTP Config: Project Working Directory/opencode.json

{
  "$schema": "https://opencode.ai/config.json",
  "mcp": {
    "NeuroREPLa": {
      "type": "remote",
      "url": "http://100.01.02.03:3001/message",
```

```
      "enabled": true
    }
  }
}

# NOTE: Is SSE supported?
```

6.2 Known Issues

- In general there are no currently reported.

7 Contributing & Development

7.1 Project Structure

- Single file: All code in `src/NeuroREPL.jl` (~1500/2000 lines)
- Modular sections: Clearly marked with comment headers
- No external files: Self-contained module

7.2 Development Workflow

```
# 1. Edit src/NeuroREPL.jl
# 2. Restart Julia REPL
# 3. Test changes
using NeuroREPL
start_server!()
```

7.3 Adding New Tools

```
# Add to get_tools() function
Tool(
    "tool_name",
    "Tool description for agents",
    Dict(
        "type" => "object",
        "properties" => Dict(
            "param" => Dict("type" => "string",
                            "description" => "Parameter description")
        ),
        "required" => ["param"]
    ),
    args -> begin
        # Tool implementation
        param = get(args, "param", "")
        execute_code(Dict("code" => "...", "show_in_repl" => true))
    end
)
```

8 License & Attribution

- License: The licence has not been decided yet, it is not specified in the codebase
- Dependencies: All dependencies are MIT or similar permissive licenses
- `Malt.jl`: MIT License (`Pluto.jl` project)
- MCP Protocol: Anthropic (protocol specification)

9 Conclusion

NeuroREPL.jl provides MCP server for AI agents to interact with Julia. Its process-isolated architecture, dual transport support, and a tool set make it suitable for:

- **AI-assisted Julia development**
- **Automated data analysis workflows**
- **Agent-driven scientific computing**
- **Interactive AI coding assistants**

The use of `Malt.jl` for worker isolation and support for both legacy and modern MCP transports ensures compatibility with a wide range of AI agent platforms while maintaining stability and performance.

References

[1] AWS, *Kiro CLI*, AWS, (2025). Keywords: #'Kiro'.

[2] Charm, *Crush*, GitHub, (2025). Keywords: #'Crush'.

[3] JuliaLang, *The Julia Programming Language*, JuliaLang, (2025). Keywords: #'julia'.

[4] MCP, *Model Context Protocol*, Model Context Protocol, (2025). Keywords: #'MCP'.

[5] Würfel, H., *MCPRepl.jl*, GitHub, (2025). Keywords: #'MCPRepl.jl'.

Appendix A: NeuroREPL v. 0.2 - Tool Reference Guide

Complete reference for all 28 MCP tools available in NeuroREPL.

Tool Categories Summary

Category	Count	Tools
Core Execution	1	execute_julia
Package Management	3	pkg_add, pkg_rm, pkg_status
Code Introspection	9	list_variables, typeinfo, methodswith, methods, names, help, which, varinfo, apropos
File Operations	3	include, pwd, cd
Performance Analysis	2	time, profile
Code Analysis	4	macroexpand, code_lowered, code_typed, code_warntype
Testing	1	test
Server Control	5	server_status, worker_status, interrupt_worker, restart_worker, reset_worker_env
TOTAL	28	

Core Execution

execute_julia

Purpose: Execute arbitrary Julia code in the isolated worker REPL.

Description: Execute arbitrary Julia code in the isolated Malt.jl worker process. This is the primary tool for running Julia commands, defining variables, loading packages, and performing computations.

Parameters: - `code` (string, required): Julia code to execute - `show_in_repl` (boolean, optional): Whether to display execution in the Julia REPL (default: true)

Implementation: `Core.eval(Main, Meta.parse(code))` in worker process

Features: - IOCapture.jl for output redirection - Process isolation via Malt.jl - Error handling with full backtrace - Optional REPL display control

Returns: Captured stdout/stderr + final expression value as string representation.

Best Practice for Agents: Use `show_in_repl: false` by default to avoid cluttering the user's REPL. The agent receives all output regardless of this setting. Only set to `true` when the user explicitly needs to see the execution (e.g., generating plots, important results). Users can globally control REPL display with `:rt` (enable) or `:rf` (disable) shortcuts.

Examples:

```julia
# Simple arithmetic
execute_julia(code: "2 + 2")
# Returns: "4"

# Activate environment and load packages
execute_julia(code: """
    using Pkg
    Pkg.activate(".")
    using DataFrames
""")

# Define variables
execute_julia(code: "x = 10; y = 20; x + y")
# Returns: "30"


# Multi-line code with output
execute_julia(code: """
    function factorial(n)
        n <= 1 ? 1 : n * factorial(n-1)
    end
    result = factorial(5)
    println("Factorial of 5 is: $result")
    result
""")
# Returns: "Factorial of 5 is: 120\n120"
```

Use Cases: - Execute any Julia code - Define functions and variables - Perform calculations - Load and use packages (after activating environment) - Create data structures - Multi-line code blocks and scripts

Notes: - First execution initializes the worker (~0.5-1 second) - Subsequent executions are fast - Output capture is automatic (stdout is included in response) - Worker state persists between calls

---

Package Management

pkg_add

Purpose: Install Julia packages to the worker environment.

Description: Add (install) Julia packages to the current environment. This modifies the Project.toml file and installs the specified packages along with their dependencies.

Parameters: - `packages` (array of strings, required): Package names to install

Implementation: `Pkg.add([package_names])`

Returns: Confirmation message and installation output.

Examples:

```julia
# Install single package
pkg_add(packages: ["Plots"])

# Install multiple packages
pkg_add(packages: ["DataFrames", "CSV", "Statistics"])

# Install specific package versions
(use execute_julia for advanced cases)

execute_julia(code: 'using Pkg;
Pkg.add(name="Plots", version="1.38")')
```

Use Cases: - Install packages needed for analysis - Add dependencies for code execution - Set up environment for specific tasks

Notes: - Packages are installed to the current environment - May take time for large packages - Requires internet connection

---

pkg_rm

Description: Remove (uninstall) Julia packages from the current environment. This modifies the Project.toml file.

Parameters: - `packages` (array of strings, required): Package names to remove

Implementation: `Pkg.rm([package_names])`

Returns: Confirmation message and removal output.

Examples:

```
# Remove single package
pkg_rm(packages: ["OldPackage"])

# Remove multiple packages
pkg_rm(packages: ["Package1", "Package2"])
```

Use Cases: - Clean up unused packages - Remove conflicting dependencies - Manage environment size

---

pkg_status

Description: Show the current environment status, including all installed packages and their versions. Equivalent to `Pkg.status()`.

Parameters: None

Implementation: `Pkg.status()`

Returns: List of installed packages with versions and status.

Examples:

```
pkg_status()
# Returns package list like:
# Status `~/Project.toml`
#   [91a5bcdd] Plots v1.38.0
#   [a93c6f00] DataFrames v1.5.0
```

Use Cases: - Check installed packages - Verify package versions - Debug dependency issues - Document environment state

---

Code Introspection

list_variables

Purpose: List all variables currently defined in the worker REPL workspace.

Description: List all variables currently defined in the Main module (REPL workspace). Useful for inspecting the current state of the REPL environment.

Parameters: None

Implementation: `Base.names(Main; all=false, imported=false)` filtered to exclude internal symbols

Returns: Newline-separated list of variable names (filtered to exclude internal symbols).

Examples:

```
# After defining some variables
execute_julia(code: "x = 1; y = 2; z = 3")
list_variables()
# Returns: "x\ny\nz\n..."
```

Use Cases: - Check what variables are defined - Inspect REPL state - Verify variable names before use - Debug variable scope issues

---

typeinfo

Description: Get comprehensive information about a Julia type, including its hierarchy, fields, and properties. Essential for understanding data structures and APIs.

Parameters: - `type_expr` (string, required): Type expression to inspect

Implementation: `supertypes(T)`, `fieldnames(T)`, `methods(T)`

Returns: Type information including supertypes, fields, and method count.

Examples:

```
# Inspect built-in type
typeinfo(type_expr: "String")

# Inspect parametric type
typeinfo(type_expr: "Vector{Int}")

# Inspect custom type
execute_julia(code: "struct Point; x::Float64; y::Float64; end")
typeinfo(type_expr: "Point")
```

Use Cases: - Understand type hierarchies - Discover available fields - Learn API structure - Debug type-related issues

---

methodswith

Description: Find all methods that have a specific type in their signature. Useful for discovering what operations are available for a type.

Parameters: - `type_expr` (string, required): Type to search for

Implementation: `InteractiveUtils.methodswith(Type)`

Returns: List of methods that accept the specified type.

Examples:

```
# Find methods for String
methodswith(type_expr: "String")

# Find methods for custom type
methodswith(type_expr: "DataFrame")
```

Use Cases: - Discover available operations for a type - Find functions that work with specific data - Explore API capabilities - Learn how to use types

---

methods

Description: List all methods of a function showing their signatures and source locations. Complements `methodswith` which finds methods using a specific type.

Parameters: - `func` (string, required): Function name (e.g., 'sort', 'push!')

Implementation: `methods(function)`

Returns: Complete list of all method signatures with file locations and line numbers.

Examples:

```
# See all sort methods
methods(func: "sort")
# Returns:
# # 12 methods for generic function "sort":
# [1] sort(v::AbstractVector) in Base.Sort at sort.jl:123
# [2] sort(v::AbstractVector; alg, lt, by, rev, order)
                                        at sort.jl:456
# ...

# Check push! methods
methods(func: "push!")
# Returns all push! methods for different container types
```

Use Cases: - Understand function capabilities - Find the right method signature - See what types a function supports - Debug method ambiguities

Difference from methodswith: - `methods(sort)` - lists all methods OF the sort function - `methodswith(Int)` - finds all methods that USE Int type

---

names

Description: List all names (functions, types, constants) exported by a module. Essential for exploring module contents and discovering available functionality.

Parameters: - `module_name` (string, required): Module to inspect (e.g., "Base", "Main", "Statistics") - `all` (boolean, optional): Include non-exported names (default: false)

Implementation: `names(Module; all=bool)`

Returns: List of names in the module.

Examples:

```
# List exported names in Base
names(module_name: "Base")

# List all names (including internal)
names(module_name: "Base", all: true)

# List names in loaded package
execute_julia(code: "using DataFrames")
names(module_name: "DataFrames")
```

Use Cases: - Explore module contents - Discover available functions - Find exported API - Learn package capabilities

---

help

Description: Get documentation for a function, type, or macro. Equivalent to Julia's `?` help system or `@doc` macro.

Parameters: - `query` (string, required): Function, type, or macro name

Implementation: `@doc symbol`

Returns: Documentation string with usage examples and descriptions.

Examples:

```
# Get help for function
help(query: "println")

# Get help for type
help(query: "String")

# Get help for macro
help(query: "@time")
```

Use Cases: - Learn function signatures - Understand function behavior - Find usage examples - Discover function parameters

---

which

Description: Show which method will be called for a given function call with specific argument types. Essential for understanding Julia's multiple dispatch.

Parameters: - `call` (string, required): Function call with arguments (e.g., 'sin(1.0)')

Implementation: `@which expression`

Returns: Method definition including file location and line number.

Examples:

```
# Check which sin method is called
which(call: "sin(1.0)")

# Check method for custom types
which(call: "myfunction(MyType())")

# Check with multiple arguments
which(call: "map(x -> x^2, [1,2,3])")
```

Use Cases: - Understand method dispatch - Debug method ambiguities - Find method definitions - Learn which implementation is used

Notes: - Shows exact method that will be called - Includes file path and line number - Helps understand type-based dispatch

---

varinfo

Description: Show comprehensive information about all variables in the workspace, including names, types, and memory sizes. Equivalent to Julia's built-in `varinfo()`.

Parameters: - `pattern` (string, optional): Regex pattern to filter variables - `sortby` (string, optional): Sort by "name", "size", or "summary" (default: "name")

Implementation: `varinfo(regex; sortby=symbol)`

Returns: Table of variables with their properties.

Examples:

```
# Show all variables
varinfo()

# Show only variables matching pattern
varinfo(pattern: "^data")

# Sort by memory size
varinfo(sortby: "size")

# Filter and sort
varinfo(pattern: "result", sortby: "size")
```

Use Cases: - Inspect workspace state - Find large variables - Debug memory usage - Check variable types

Notes: - Shows name, size, and summary for each variable - Useful for memory profiling - Can filter with regex patterns

---

apropos

Description: Search Julia documentation by keyword to discover related functions, types, and modules. Essential for exploring the Julia ecosystem and finding functions you don't know exist.

Parameters: - `keyword` (string, required): Keyword to search for in documentation

Implementation: `apropos(keyword)`

Returns: List of all functions, types, and modules matching the keyword.

Examples:

```
# Find all sorting-related functions
apropos(keyword: "sort")
# Returns: Base.sort, Base.sort!, Base.sortperm,
           Base.issorted, Base.partialsort, ...

# Discover plotting functions
apropos(keyword: "plot")
# Returns: All plot-related functions from loaded packages

# Find array functions
apropos(keyword: "array")
# Returns: Array, AbstractArray, similar, reshape, ...
```

Use Cases: - Discover functions related to a task - Find alternative implementations - Explore package functionality - Learn what's available in loaded packages

---

File Operations

include

Description: Load and execute a Julia source file. This is Julia's standard way to include external code files into the current session.

Parameters: - `path` (string, required): Path to Julia file (relative or absolute)

Implementation: `include(path)` in worker

Returns: Result of executing the file, or error if file not found.

Examples:

```julia
# Load local file
include(path: "script.jl")

# Load file with absolute path
include(path: "/home/user/code/utils.jl")

# Load file from subdirectory
include(path: "./src/helpers.jl")
```

Use Cases: - Load utility functions - Execute setup scripts - Import custom code - Organize code into modules

Notes: - File is executed in Main module - Variables and functions become available immediately - Relative paths are relative to current working directory

---

pwd

Description: Get the current working directory. Equivalent to Julia's built-in `pwd()` function.

Parameters: None

Implementation: `pwd()`

Returns: Current working directory path as string.

Examples:

```julia
pwd()
# Returns: "/home/user/project"
```

Use Cases: - Check current location - Verify directory before file operations - Debug path issues

---

cd

Description: Change the current working directory. Equivalent to Julia's built-in `cd()` function.

Parameters: - `path` (string, required): Directory path to change to

Implementation: `cd(path); pwd()`

Returns: New current directory path.

Examples:

```julia
# Change to absolute path
cd(path: "/home/user/project")

# Change to relative path
cd(path: "../other-project")

# Change to home directory
cd(path: "~")
```

Use Cases: - Navigate to project directory - Access files in different locations - Set up working environment

Notes: - Affects all subsequent file operations - Use absolute paths for reliability - Changes persist for the session

---

Performance Analysis

time

Description: Measure execution time, memory allocations, and compilation time for an expression. Equivalent to Julia's `@time` macro.

Parameters: - `expr` (string, required): Expression to time

Implementation: `@time expression`

Returns: Execution time, allocation count, and memory usage, plus the expression result.

Examples:

```
# Time simple expression
time(expr: "sum(1:1000000)")

# Time function call
time(expr: "factorial(20)")

# Time complex operation
time(expr: "sort(rand(10000))")
```

Use Cases: - Measure performance - Compare implementations - Identify slow operations - Check memory allocations

Notes: - First run includes compilation time - Run twice for accurate timing - Shows allocations (important for performance) - Memory allocations often indicate performance issues

---

profile

Description: Profile code execution to identify performance bottlenecks. Shows where time is spent in your code. Uses Julia's built-in `@profile` macro.

Parameters: - `expr` (string, required): Expression to profile - `show_flat` (boolean, optional): Show flat profile format (default: true)

Implementation: `Profile.clear(); @profile expr; Profile.print()`

Returns: Profile report showing time spent in each function.

Examples:

```
# Profile simple expression
profile(expr: "sum(1:1000000)")

# Profile function with tree view
profile(expr: "my_complex_function()", show_flat: false)

# Profile with flat view (default)
profile(expr: "process_data(df)")
```

Use Cases: - Find performance bottlenecks - Optimize slow code - Understand execution flow - Identify hot spots

Notes: - Requires multiple samples for accuracy - Flat view shows functions sorted by time - Tree view shows call hierarchy - Run code multiple times for better profiling

---

Code Analysis

macroexpand

Description: Expand a macro expression to see the code it generates. Essential for understanding macro behavior and debugging macro-related issues.

Parameters: - `expr` (string, required): Macro expression to expand

Implementation: `macroexpand(Main, :(expression))`

Returns: Expanded code representation.

Examples:

```
# Expand @time macro
macroexpand(expr: "@time 1+1")

# Expand custom macro
execute_julia(code: "macro double(x) :(\$x * 2) end")
macroexpand(expr: "@double 5")
```

Use Cases: - Understand macro transformations - Debug macro behavior - Learn macro implementation - Verify macro correctness

---

code_lowered

Description: Show the lowered (desugared) intermediate representation of code. This is the first stage of Julia's compilation pipeline, useful for understanding how high-level syntax is transformed.

Parameters: - `expr` (string, required): Expression to lower

Implementation: `@code_lowered expression`

Returns: Lowered code representation.

Examples:

```
# Show lowered code for simple expression
code_lowered(expr: "sin(1.0)")

# Show lowered code for function call
code_lowered(expr: "map(x -> x^2, [1,2,3])")
```

Use Cases: - Understand code transformations - Debug complex expressions - Learn Julia internals - Optimize code structure

Notes: - Advanced feature for performance tuning - Shows desugared syntax - Useful for compiler understanding

---

code_typed

Description: Show the type-inferred intermediate representation of code. This shows what types Julia's compiler has inferred for all variables and expressions, essential for performance optimization.

Parameters: - `expr` (string, required): Expression to analyze

Implementation: `@code_typed expression`

Returns: Type-inferred code representation.

Examples:

```
# Show type inference for expression
code_typed(expr: "sin(1.0)")
```

```
# Analyze function with type inference
execute_julia(code: "f(x) = x^2 + 2x + 1")
code_typed(expr: "f(5)")
```

Use Cases: - Verify type stability - Optimize performance - Debug type inference issues - Understand compiler behavior

Notes: - Critical for performance optimization - Shows inferred types for all operations - Helps identify type instabilities

---

code_warntype

Description: Show type stability analysis with warnings for performance debugging. Highlights type instabilities that cause performance problems. More readable than `code_typed` for finding performance issues.

Parameters: - `call` (string, required): Function call with arguments (e.g., 'my_function(1, 2.0)')

Implementation: `@code_warntype expression`

Returns: Type-inferred code with warnings about type instabilities highlighted.

Examples:

```
# Check type stability
code_warntype(call: "my_function(1, 2.0)")
# Shows:

# WARNING: type unstable (bad for performance)
# Body::Union{Int64, Float64}
# Body::Int64                    # Good: type stable

# Debug performance issue
code_warntype(call: "process_data(data)")
# Highlights where types are not inferred correctly

# Verify optimization
code_warntype(call: "optimized_function(x)")
# Confirms all types are stable (no warnings)
```

Use Cases: - Find performance bottlenecks - Debug type instabilities - Verify optimizations worked - Understand why code is slow

Type Stability: - **Stable** (good): Function always returns same type - **Unstable** (bad): Return type depends on values, not types - Unstable code is much slower (type inference fails)

Difference from code_typed: - `code_typed` - raw type inference output - `code_warntype` - highlights problems with warnings (easier to read)

Testing

test

Description: Run package tests using Julia's test framework. Equivalent to `Pkg.test()`.

Parameters: - `package` (string, optional): Package name to test (tests current project if omitted)

Implementation: `Pkg.test(package)` or `Pkg.test()`

Returns: Test results showing passed/failed tests.

Examples:

```
# Test current project
test()

# Test specific package
test(package: "DataFrames")

# Test after making changes
test(package: "MyPackage")
```

Use Cases: - Verify package functionality - Run test suites - Check after modifications - Continuous integration

Notes: - Runs tests in test/runtests.jl - Shows detailed test results - May take time for large test suites - Requires test dependencies to be installed

---

Server Control

server_status

Description: Show comprehensive MCP server status including transport details, environment paths, request statistics, and settings.

Parameters: None

Implementation: Custom status reporting function

Returns: Formatted string with complete server information.

Examples:

```
server_status()
```

```
Returns:
✓ NeuroREPL:         MCP Server running on http://100.01.02.03:3001
   Protocol:           MCP 2025-03-26+
   Transport:          Streamable HTTP
   Endpoint:           POST /message
   Features:           Request/response, session support
   Status:             ✓ Active
   • Total requests:   42
   • Last request:     2.3s ago
   Agent Tools:        28
   SERVER_IP:          100.01.02.03
   SERVER_PORT:        3001
   NeuroREPL env:      /path/to/NeuroREPL
   Worker env:         /path/to/NeuroREPL/worker_env
   Verbose logging:    true
   Show code in REPL:  true
   Capture output:     true"
```

Use Cases: - Get complete server overview - Check environment paths - Verify server configuration - Debug connection issues - Monitor request statistics

---

worker_status

Description: Check the health and status of the worker process.

Parameters: None

Implementation: `Malt.isrunning(worker)`

Returns: String describing worker status: “Not initialized”, “Running (healthy)”, or “Dead (not running)”.

Examples:

```
worker_status()
# Returns: "Worker: Running (healthy)"
```

Use Cases: - Verify worker is responsive before executing code - Debug connection issues - Check if worker needs restart - Diagnose crashes

---

interrupt_worker

Description: Interrupt long-running code execution in the worker process.

Parameters: None

Implementation: `Malt.interrupt(worker)`

Returns: Confirmation message or error if worker not running.

Examples:

```
interrupt_worker()
# Returns: "Interrupt signal sent to worker"
```

Use Cases: - Stop infinite loops or long computations - Recover from unresponsive code - Cancel operations that are taking too long

Notes: - Cross-platform support (including Windows) - Sends SIGINT to worker process - Non-blocking operation - Worker remains alive and usable after interrupt

restart_worker

Purpose: Restart the Julia worker process for a clean session.

Description: Restart the Malt.jl worker process to get a completely clean session. All variables, functions, loaded packages, and state are cleared. The worker process is stopped and a new one is spawned.

Parameters: None

Implementation: `Malt.stop(worker); worker = Malt.Worker()`

Features: - Clears all variables and state - Keeps packages in worker_env - Preserves package installations

Returns: Confirmation message.

Examples:

```
# After doing some work
execute_julia(code: "x = 100; using DataFrames")

# Restart for clean slate
restart_worker()
# Returns: "Worker restarted. All variables and modules cleared."

# x and DataFrames are no longer available
execute_julia(code: "x")
# Returns: Error - x not defined
```

Use Cases: - Start fresh after testing - Clear memory from large computations - Resolve package conflicts - Test code in clean environment - Debug state-related issues - Clear loaded C libraries - Recover from unresponsive worker

Notes: - Takes ~0.5-1 seconds to spawn new Malt worker (faster than Distributed.jl) - All state is completely cleared (like restarting Julia) - Worker starts with clean environment - must reactivate project - Modules can be redefined

after restart - C libraries are unloaded - Server continues running (only worker restarts)

reset_worker_env

Description: Delete the worker environment and restart with a completely fresh slate. This removes all installed packages and clears all state.

Parameters: None

Implementation: `rm(worker_env, recursive=true); restart_worker!()`

Features: - Removes all installed packages - Deletes Project.toml and Manifest.toml - Complete environment reset

Returns: Confirmation message.

Examples:

```
reset_worker_env()
# Returns: "Worker environment reset, worker_env directory deleted,
# all packages removed, fresh session started."
```

Use Cases: - Resolve package conflicts - Start with clean environment after testing - Remove all installed packages - More thorough than `restart_worker` (which keeps packages)

Notes: - This is more aggressive than `restart_worker` - it deletes the entire `worker_env` directory and recreates it - Use when you need a truly clean slate - Requires reinstalling all packages - slower than restart_worker - All packages must be reinstalled after reset - Takes longer than restart_worker due to package reinstallation

---

Appendix B: NeuroREPL v. 0.2 - Screenshots

NeuroREPL Starting Sequence - Part 1/2

```
✓ NeuroREPL loading...
  Wait for explicit confirmation that NeuroREPL is READY

✓ Checking NeuroREPL dependencies...
  Dependencies already installed

✓ Activating NeuroREPL environment...
  NeuroREPL environment activated

✓ NeuroREPL Server starting... (127.0.0.1:3001)
  Ensure correct IP/PORT configuration in case of connection issues
  See <src/NeuroREPL.jl> for defaults, overridden by <nr.jl>


               _
   _       _ _(_)_     |  Documentation: https://docs.julialang.org
  (_)     | (_) (_)    |
   _ _   _| |_  __ _   |  Type "?" for help, "]?" for Pkg help.
  | | | | | | |/ _` |  |
  | | |_| | | | (_| |  |  Version 1.12.0 (2025-10-07)
 _/ |\__'_|_|_|\__'_|  |  Official https://julialang.org release
|__/                   |

julia> [ Info: Listening on: 127.0.0.1:3001, thread id: 1
✓ Setting NeuroREPL Worker mode...
[Worker 2739554]:   Activating project at `/mnt/data/_NeuroREPL/worker_env`
  NeuroREPL Worker mode initialized

✓ NeuroREPL Server Status
  Server:                 MCP Server running on http://127.0.0.1:3001
    SERVER_IP:            127.0.0.1
    SERVER_PORT:          3001
    Verbose Logging:      false
    Show Code in REPL:    true
    Capture Output:       true
    Agent Tools:          28
  Protocol:               MCP
    Transports:           SSE (2024-11-05), Streamable HTTP (2025-03-26)
    SSE Endpoints:        GET /sse → POST /sse/message
    HTTP Endpoint:        POST /message
    Health Check:         GET /
  Transport Status:       Dual transport, session support
    SSE:                  ○ No connection yet
    SSE keepalive:        false (5.0s)
    HTTP:                 ○ No requests yet
  Request Statistics:
    No requests yet
  Environments:           Agent development environment instantiated in: worker_env
    NeuroREPL:            /mnt/data/_NeuroREPL
    Worker / Agent:       /mnt/data/_NeuroREPL/worker_env
```

NeuroREPL Starting Sequence - Part 2/2

```
√ NeuroREPL Agent Tools: 28
  Category:            Count:  Tools:
  Core Execution         1     execute_julia
  Package Management     3     pkg_add, pkg_rm, pkg_status
  Code Introspection     9     list_variables, typeinfo,
                               methodswith, methods, names, help,
                               which, varinfo, apropos
  File Operations        3     include, pwd, cd
  Performance Analysis   2     time, profile
  Code Analysis          4     macroexpand, code_lowered,
                               code_typed, code_warntype
  Testing                1     test
  Server Control         5     server_status, worker_status,
                               interrupt_worker, restart_worker,
                               reset_worker_env

√ NeuroREPL Shortcuts
  :w         - Switch to worker mode (from server)
  :s         - Switch to server mode (from worker)
  :ms        - mode_status()
  :ss        - server_status()
  :ws        - worker_status!()
  :iw        - init_worker!()
  :i!        - interrupt_worker!()
  :r!        - restart_worker!()
  :re!       - reset_worker_env!()
  :start!    - start_server!()
  :stop!     - stop_server!()
  :vt/:vf    - verbose_server!(true/false)
  :rt/:rf    - show_repl!(true/false)
  :at/:af    - agent_capture!(true/false)
  :ka        - send_sse_keepalive()
  :hs        - Show shortcuts help
  :hat       - Show agent tools help
  :hnr       - Show full help
  :q!        - Exit Julia

√ READY
  NeuroREPL v. 0.2 loaded
  Press Enter to start coding in Julia

  |\| |_ |_| |\ () |\ |_ |¯ |_ . _| |_

julia>
```

Appendix C: NeuroREPL v. 0.1 (Legacy Version) - Prompt Examples

What packages are installed in the current NeuroREPL environment?

```
❯ ./start-NeuroREPL
Starting NeuroREPL with MCP server...

  Activating project at `~/data/NeuroREPL`
  Activating project at `~/data/NeuroREPL`
   Resolving package versions...
     Project No packages added to or removed from `~/data/NeuroREPL/Project.toml`
    Manifest No packages added to or removed from `~/data/NeuroREPL/Manifest.toml`
✓ NeuroREPL with MCP server
  Server starting on port 3000...


               _
   _       _ _(_)_     |  Documentation: https://docs.julialang.org
  (_)     | (_) (_)    |
   _ _   _| |_  __ _   |  Type "?" for help, "]?" for Pkg help.
  | | | | | | |/ _` |  |
  | | |_| | | | (_| |  |  Version 1.12.1 (2025-10-17)
 _/ |\__'_|_|_|\__'_|  |  Official https://julialang.org release
|__/                   |

julia> [ Info: Listening on: 127.0.0.1:3000, thread id: 1
✓ NeuroREPL MCP Server running on http://127.0.0.1:3000
  Tools: 20

  Settings:
    • Verbose server logging:   OFF → verbose_server!(true/false)
    • Show code in REPL:        ON  → show_repl!(true/false)
    • Capture output for agent: ON  → agent_capture!(true/false)
julia> using Pkg; Pkg.status()
Status `~/data/NeuroREPL/Project.toml`
  [cd3eb016] HTTP v1.10.19
  [682c06a0] JSON v1.3.0
nothing
```

```
❯ kiro-cli chat
✓ NeuroREPL loaded in 2.58 s

KIRO

Did you know?
Get notified whenever Kiro CLI finishes responding. Just run
kiro-cli settings chat.enableNotifications true

Model: Auto (/model to change) | Plan: KIRO FREE (/usage for more detail)

> What packages are installed in the current NeuroREPL environment?

> I'll check the installed packages in the current Julia environment.
Running tool status with the param (from mcp server: NeuroREPL)
 ⋮  {}

Allow this action? Use 't' to trust (always allow) this tool for the session. [y/n/t]:

> t

 - Completed in 3.639s

> The current NeuroREPL environment has 2 packages installed:

- **HTTP** v1.10.19 - For HTTP client/server functionality
- **JSON** v1.3.0 - For JSON parsing and serialization

These are defined in the Project.toml file in your current directory.
 ▸ Credits: 0.04 • Time: 29s

>
```

What tools are available for Julia NeuroREPL MCP - please check directly in Julia?

```
[surface] ./start-NeuroREPL ~/d/m/M/NeuroREPL                                   SCROLL:  0/96
ption" => "Show execution in Julia REPL (default: true)")), "required" => ["code"], "type" => "object"), Main.NeuroREPL.exec
ute_code), Main.NeuroREPL.Tool("list_variables", "List all variables in REPL", Dict{String, Any}("properties" => Dict{Any, A
ny}(), "required" => Any[], "type" => "object"), Main.NeuroREPL.var"#get_tools##0#get_tools##1"()), Main.NeuroREPL.Tool("pkg
_add", "Add Julia packages to the current environment", Dict{String, Any}("properties" => Dict{String, Dict{String, Any}}("p
ackages" => Dict("items" => Dict("type" => "string"), "type" => "array", "description" => "Array of package names to add")),
 "required" => ["packages"], "type" => "object"), Main.NeuroREPL.var"#get_tools##4#get_tools##5"()), Main.NeuroREPL.Tool("pk
g_rm", "Remove Julia packages from the current environment", Dict{String, Any}("properties" => Dict{String, Dict{String, Any
}}("packages" => Dict("items" => Dict("type" => "string"), "type" => "array", "description" => "Array of package names to re
move")), "required" => ["packages"], "type" => "object"), Main.NeuroREPL.var"#get_tools##8#get_tools##9"()), Main.NeuroREPL.
Tool("typeinfo", "Get type information: hierarchy, fields, and properties", Dict{String, Any}("properties" => Dict("type_exp
r" => Dict("type" => "string", "description" => "Type expression (e.g., 'String', 'Vector{Int}')")), "required" => ["type_ex
pr"], "type" => "object"), Main.NeuroREPL.var"#get_tools##12#get_tools##13"()), Main.NeuroREPL.Tool("methodswith", "Find all
 methods that have a type in their signature", Dict{String, Any}("properties" => Dict("type_expr" => Dict("type" => "string"
, "description" => "Type to search for (e.g., 'String', 'Int')")), "required" => ["type_expr"], "type" => "object"), Main.Ne
uroREPL.var"#get_tools##14#get_tools##15"()), Main.NeuroREPL.Tool("names", "List all names in a module", Dict{String, Any}("
properties" => Dict("module_name" => Dict("type" => "string", "description" => "Module name (e.g., 'Base', 'Main')"), "all"
=> Dict("type" => "boolean", "description" => "Include non-exported names (default: false)")), "required" => ["module_name"]
, "type" => "object"), Main.NeuroREPL.var"#get_tools##16#get_tools##17"()), Main.NeuroREPL.Tool("macroexpand", "Expand a mac
ro expression to see what it generates", Dict{String, Any}("properties" => Dict("expr" => Dict("type" => "string", "descript
ion" => "Macro expression (e.g., '@time 1+1')")), "required" => ["expr"], "type" => "object"), Main.NeuroREPL.var"#get_tools
##18#get_tools##19"()), Main.NeuroREPL.Tool("help", "Get help documentation for a function or type", Dict{String, Any}("prop
erties" => Dict("query" => Dict("type" => "string", "description" => "Function or type name (e.g., 'println', 'String')")),
"required" => ["query"], "type" => "object"), Main.NeuroREPL.var"#get_tools##20#get_tools##21"()), Main.NeuroREPL.Tool("stat
us", "Show environment and package status", Dict{String, Any}("properties" => Dict{Any, Any}(), "required" => Any[], "type"
=> "object"), Main.NeuroREPL.var"#get_tools##22#get_tools##23"()), Main.NeuroREPL.Tool("code_lowered", "Show lowered (desuga
red) code representation", Dict{String, Any}("properties" => Dict("expr" => Dict("type" => "string", "description" => "Expre
ssion to lower (e.g., 'sin(1.0)')")), "required" => ["expr"], "type" => "object"), Main.NeuroREPL.var"#get_tools##24#get_too
ls##25"()), Main.NeuroREPL.Tool("code_typed", "Show type-inferred code representation", Dict{String, Any}("properties" => Di
ct("expr" => Dict("type" => "string", "description" => "Expression to analyze (e.g., 'sin(1.0)')")), "required" => ["expr"],
 "type" => "object"), Main.NeuroREPL.var"#get_tools##26#get_tools##27"()), Main.NeuroREPL.Tool("include", "Load and execute
a Julia source file", Dict{String, Any}("properties" => Dict("path" => Dict("type" => "string", "description" => "Path to Ju
lia file (relative or absolute)")), "required" => ["path"], "type" => "object"), Main.NeuroREPL.var"#get_tools##28#get_tools
##29"()), Main.NeuroREPL.Tool("pwd", "Get current working directory", Dict{String, Any}("properties" => Dict{Any, Any}(), "r
equired" => Any[], "type" => "object"), Main.NeuroREPL.var"#get_tools##30#get_tools##31"()), Main.NeuroREPL.Tool("cd", "Chan
ge current working directory", Dict{String, Any}("properties" => Dict("path" => Dict("type" => "string", "description" => "D
irectory path to change to")), "required" => ["path"], "type" => "object"), Main.NeuroREPL.var"#get_tools##32#get_tools##33"
()), Main.NeuroREPL.Tool("which", "Show which method will be called for given arguments", Dict{String, Any}("properties" =>
Dict("call" => Dict("type" => "string", "description" => "Function call with arguments (e.g., 'sin(1.0)')")), "required" =>
["call"], "type" => "object"), Main.NeuroREPL.var"#get_tools##34#get_tools##35"()), Main.NeuroREPL.Tool("varinfo", "Show inf
ormation about all variables (names, types, sizes)", Dict{String, Any}("properties" => Dict("sortby" => Dict("type" => "stri
ng", "description" => "Sort by: name, size, or summary (default: name)"), "pattern" => Dict("type" => "string", "description
" => "Regex pattern to filter variables (optional)")), "required" => Any[], "type" => "object"), Main.NeuroREPL.var"#get_too
ls##36#get_tools##37"()), Main.NeuroREPL.Tool("test", "Run package tests", Dict{String, Any}("properties" => Dict("package"
=> Dict("type" => "string", "description" => "Package name to test (optional, tests current project if omitted)")), "require
d" => Any[], "type" => "object"), Main.NeuroREPL.var"#get_tools##38#get_tools##39"()), Main.NeuroREPL.Tool("time", "Measure
execution time and memory allocations", Dict{String, Any}("properties" => Dict("expr" => Dict("type" => "string", "descripti
on" => "Expression to time (e.g., 'sum(1:1000000)')")), "required" => ["expr"], "type" => "object"), Main.NeuroREPL.var"#get
_tools##40#get_tools##41"()), Main.NeuroREPL.Tool("profile", "Profile code execution to find performance bottlenecks", Dict{
String, Any}("properties" => Dict("show_flat" => Dict("type" => "boolean", "description" => "Show flat profile (default: tru
e)"), "expr" => Dict("type" => "string", "description" => "Expression to profile")), "required" => ["expr"], "type" => "obje
ct"), Main.NeuroREPL.var"#get_tools##42#get_tools##43"())]
julia> for tool in NeuroREPL.get_tools()
    println("• $(tool.name): $(tool.description)")
end
• execute_julia: Execute Julia code in the REPL
• list_variables: List all variables in REPL
• pkg_add: Add Julia packages to the current environment
• pkg_rm: Remove Julia packages from the current environment
• typeinfo: Get type information: hierarchy, fields, and properties
• methodswith: Find all methods that have a type in their signature
• names: List all names in a module
• macroexpand: Expand a macro expression to see what it generates
• help: Get help documentation for a function or type
• status: Show environment and package status
• code_lowered: Show lowered (desugared) code representation
• code_typed: Show type-inferred code representation
• include: Load and execute a Julia source file
• pwd: Get current working directory
• cd: Change current working directory
• which: Show which method will be called for given arguments
• varinfo: Show information about all variables (names, types, sizes)
• test: Run package tests
• time: Measure execution time and memory allocations
• profile: Profile code execution to find performance bottlenecks
nothing
```

```
[surface] kiro-cli chat ~/d/NeuroREPL
> What tools are available for Julia NeuroREPL MCP - please check directly in Julia?

> I'll check what's available in the Julia environment by looking at the Main module and any loaded packages.
Running tool execute_julia with the param (from mcp server: NeuroREPL)
 ⋮  {
 ⋮    "code": "names(Main)"
 ⋮  }

Allow this action? Use 't' to trust (always allow) this tool for the session. [y/n/t]:

> t

 - Completed in 0.544s

Running tool execute_julia with the param (from mcp server: NeuroREPL)
 ⋮  {
 ⋮    "code": "names(NeuroREPL, all=true)"
 ⋮  }
 - Completed in 0.30s

Running tool execute_julia with the param (from mcp server: NeuroREPL)
 ⋮  {
 ⋮    "code": "NeuroREPL.get_tools()"
 ⋮  }
 - Completed in 2.567s

Running tool execute_julia with the param (from mcp server: NeuroREPL)
 ⋮  {
 ⋮    "code": "for tool in NeuroREPL.get_tools()\n    println(\"• $(tool.name): $(tool.description)\")\nend"
 ⋮  }
 - Completed in 0.69s

> The Julia NeuroREPL MCP server provides 20 tools:

Code Execution & Development:
- execute_julia: Execute Julia code in the REPL
- include: Load and execute a Julia source file
- list_variables: List all variables in REPL

Package Management:
- pkg_add: Add Julia packages to the current environment
- pkg_rm: Remove Julia packages from the current environment
- status: Show environment and package status
- test: Run package tests

Type & Method Inspection:
- typeinfo: Get type information: hierarchy, fields, and properties
- methodswith: Find all methods that have a type in their signature
- names: List all names in a module
- which: Show which method will be called for given arguments

Code Analysis:
- code_lowered: Show lowered (desugared) code representation
- code_typed: Show type-inferred code representation
- macroexpand: Expand a macro expression to see what it generates

Performance:
- time: Measure execution time and memory allocations
- profile: Profile code execution to find performance bottlenecks

Utilities:
- help: Get help documentation for a function or type
- varinfo: Show information about all variables (names, types, sizes)
- pwd: Get current working directory
- cd: Change current working directory
 ▸ Credits: 0.16 • Time: 25s

>
```

What transport protocol does NeuroREPL use? [It is the MCP (Model Context Protocol) server that provides the Julia-related tools you have access to.]

```
[surface] ./start-NeuroREPL ~/d/m/M/NeuroREPL                                   SCROLL:  0/172
julia>
```

```
[surface] kiro-cli chat ~/d/NeuroREPL
> What transport protocol does NeuroREPL use? [It is the MCP (Model Context Protocol) server that provides the Julia-related
tools you have access to.]

> I'll check the NeuroREPL source code to see what transport protocol it uses.
Reading directory: /home/funky/data/NeuroREPL (using tool: read, max depth: 0)
 ✓ Successfully read directory /home/funky/data/NeuroREPL (7 entries)
 - Completed in 0.0s

Reading directory: /home/funky/data/NeuroREPL/src (using tool: read, max depth: 0)
 ✓ Successfully read directory /home/funky/data/NeuroREPL/src (1 entries)
 - Completed in 0.0s

Searching: /home/funky/data/NeuroREPL/src/NeuroREPL.jl for pattern: transport|protocol|sse|stdio|http (using tool: read)
 ✓ Found 0 matches for pattern 'transport|protocol|SSE|stdio|HTTP' in /home/funky/data/NeuroREPL/src/NeuroREPL.jl
 - Completed in 0.0s

Reading file: /home/funky/data/NeuroREPL/src/NeuroREPL.jl, from line 1 to 100 (using tool: read)
 ✓ Successfully read 3119 bytes from /home/funky/data/NeuroREPL/src/NeuroREPL.jl
 - Completed in 0.0s

Searching: /home/funky/data/NeuroREPL/src/NeuroREPL.jl for pattern: sse|server-sent events|event-stream (using tool: read)
 ✓ Found 0 matches for pattern 'SSE|Server-Sent Events|event-stream' in /home/funky/data/NeuroREPL/src/NeuroREPL.jl
 - Completed in 0.0s

Reading file: /home/funky/data/NeuroREPL/src/NeuroREPL.jl, from line 500 to 600 (using tool: read)
 ✓ Successfully read 3730 bytes from /home/funky/data/NeuroREPL/src/NeuroREPL.jl
 - Completed in 0.0s

Reading file: /home/funky/data/NeuroREPL/src/NeuroREPL.jl, from line 600 to 700 (using tool: read)
 ✓ Successfully read 4272 bytes from /home/funky/data/NeuroREPL/src/NeuroREPL.jl
 - Completed in 0.0s

> Based on the source code, NeuroREPL uses SSE (Server-Sent Events) over HTTP as its transport protocol for MCP.

Specifically:
- The server listens on HTTP port 3000 (by default)
- It provides an SSE endpoint at /sse or / for GET requests
- Clients connect to the SSE stream to receive responses
- Clients send JSON-RPC requests via POST to /message
- The server queues responses through an internal channel and sends them back via the SSE stream as event: message events

This is a bidirectional communication pattern where:
1. SSE provides server-to-client streaming (for responses)
2. HTTP POST provides client-to-server messaging (for requests)

The server also supports a direct JSON-RPC endpoint at / for POST requests (non-SSE clients), but the primary MCP transport
is SSE.
 ▸ Credits: 0.32 • Time: 31s

>
```

Is the NeuroREPL server protected by any network-security measures, or does it run on an unsecured localhost port?

```
[surface] ./start-NeuroREPL ~/d/m/M/NeuroREPL                                   SCROLL:  0/172
julia>
```

```
[surface] kiro-cli chat ~/d/NeuroREPL
> Is the NeuroREPL server protected by any network-security measures, or does it run on an unsecured localhost port?

Searching: /home/funky/data/NeuroREPL/src/NeuroREPL.jl for pattern: start_server|listen|localhost|127.0.0.1|0.0.0.0|auth|sec
urity|token|password (using tool: read)
 ✓ Found 0 matches for pattern 'start_server|listen|localhost|127.0.0.1|0.0.0.0|auth|security|token|password' in /home/funky
/data/NeuroREPL/src/NeuroREPL.jl
 - Completed in 0.0s

Reading file: /home/funky/data/NeuroREPL/src/NeuroREPL.jl, from line 700 to 800 (using tool: read)
 ✓ Successfully read 1974 bytes from /home/funky/data/NeuroREPL/src/NeuroREPL.jl
 - Completed in 0.0s

> The NeuroREPL server runs unsecured on localhost (127.0.0.1) with no network security measures:

- Binds to 127.0.0.1:3000 (localhost only)
- No authentication (no tokens, passwords, or API keys)
- No authorization checks
- No TLS/SSL encryption
- No request validation beyond basic JSON-RPC parsing
- Anyone with local access can connect and execute arbitrary Julia code

The only "security" is that it's bound to localhost (127.0.0.1), meaning it's not accessible from external networks by
default. However, any process running on the same machine can connect to it and execute arbitrary Julia code through the
execute_julia tool.

This is typical for local development tools and MCP servers that are designed to run on a developer's machine, where the
security boundary is the operating system's user isolation.
 ▸ Credits: 0.14 • Time: 14s

>
```

Let’s test NeuroREPL. Please add 2 + 2 in Julia.

```
[surface] ./start-NeuroREPL ~/d/m/M/NeuroREPL                    SCROLL:  0/172
julia> julia> 2 + 2
4
```

```
[surface] kiro-cli chat ~/d/NeuroREPL
> Let's test NeuroREPL. Please add 2 + 2 in Julia.

Running tool execute_julia with the param (from mcp server: NeuroREPL)
  {
    "code": "2 + 2"
  }
 - Completed in 0.102s

> The result is 4.
 ▸ Credits: 0.06 • Time: 5s

>
```

Plot a heatmap (8 x 8) in NeuroREPL via MCP with UnicodePlots.jl and show me the result. Remember to check if the UnicodePlots.jl is installed in the current environment.

```
[surface] ./start-NeuroREPL ~/d/NeuroREPL                                   SCROLL:  0/1346
julia> julia> using Pkg; Pkg.status()
Status `~/data/NeuroREPL/Project.toml`
  [cd3eb016] HTTP v1.10.19
  [682c06a0] JSON v1.3.0
nothing
julia> using Pkg; Pkg.add(["UnicodePlots"])
   Resolving package versions...
    Updating `~/data/NeuroREPL/Project.toml`
  [b8865327] + UnicodePlots v3.8.1
    Updating `~/data/NeuroREPL/Manifest.toml`
  [66dad0bd] + AliasTables v1.1.3
  [35d6a980] + ColorSchemes v3.31.0
  [3da002f7] + ColorTypes v0.12.1
  [c3611d14] + ColorVectorSpace v0.11.0
  [5ae59095] + Colors v0.13.1
  [d38c429a] + Contour v0.6.3
  [a8cc5b0e] + Crayons v4.1.1
  [9a962f9c] + DataAPI v1.16.0
  [864edb3b] + DataStructures v0.19.3
  [ffbed154] + DocStringExtensions v0.9.5
  [53c48c17] + FixedPointNumbers v0.8.5
  [92d709cd] + IrrationalConstants v0.2.6
  [2ab3a3ac] + LogExpFunctions v0.3.29
  [299715c1] + MarchingCubes v0.1.11
  [e1d29d7a] + Missings v1.2.0
  [77ba4419] + NaNMath v1.1.3
  [bac558e1] + OrderedCollections v1.8.1
  [43287f4e] + PtrArrays v1.3.0
  [189a3867] + Reexport v1.2.2
  [ae029012] + Requires v1.3.1
  [a2af1166] + SortingAlgorithms v1.2.2
  [90137ffa] + StaticArrays v1.9.15
  [1e83bf80] + StaticArraysCore v1.4.4
  [10745b16] + Statistics v1.11.1
  [82ae8749] + StatsAPI v1.7.1
  [2913bbd2] + StatsBase v0.34.8
  [62fd8b95] + TensorCore v0.1.1
  [b8865327] + UnicodePlots v3.8.1
  [37e2e46d] + LinearAlgebra v1.12.0
  [2f01184e] + SparseArrays v1.12.0
  [e66e0078] + CompilerSupportLibraries_jll v1.3.0+1
  [4536629a] + OpenBLAS_jll v0.3.29+0
  [05823500] + OpenLibm_jll v0.8.7+0
  [bea87d4a] + SuiteSparse_jll v7.8.3+2
  [8e850b90] + libblastrampoline_jll v5.15.0+0
nothing
julia> using UnicodePlots; heatmap(rand(8, 8))
```

```
[surface] kiro-cli chat ~/d/NeuroREPL_test                                  SCROLL:  0/204
> Plot a heatmap (8 z 8) in NeuroREPL via MCP with UnicodePlots.jl and show me the result.
  Remember to check if the UnicodePlots.jl is installed in the current environment.

> I'll check the package status and create the heatmap:
Running tool status with the param (from mcp server: NeuroREPL)
 ⋮  {}
 - Completed in 0.42s

Running tool pkg_add with the param (from mcp server: NeuroREPL)
 ⋮  {
 ⋮    "packages": [
 ⋮      "UnicodePlots"
 ⋮    ]
 ⋮  }
 - Completed in 1.621s

Running tool execute_julia with the param (from mcp server: NeuroREPL)
 ⋮  {
 ⋮    "code": "using UnicodePlots; heatmap(rand(8, 8))"
 ⋮  }
 - Completed in 0.47s

> Here's the 8x8 heatmap:

The heatmap shows an 8x8 matrix with random values, where darker blocks represent higher values (up to 0.9) and lighter
blocks represent lower values (down to 4e⁻⁵).
 ▸ Credits: 0.13 • Time: 14s

>
```

Appendix D: NeuroREPL v. 0.2 - Limitations & Potential Enhancements (UNEDITED)

Note: This part is unedited - it is a raw model output.

1. No LSP Integration

NeuroREPL does not provide Language Server Protocol (LSP) features for the worker REPL environment. This means:

Missing capabilities: - No code completion for variables/functions defined in the worker session - No go-to-definition for code executed in the REPL - No find-references for symbols in the worker workspace - No hover documentation for runtime-defined functions - No signature help for custom functions

What agents must use instead: - `help(query)` - Get documentation for built-in functions/types - `which(call)` - Show which method will be called - `methods(func)` - List all methods of a function - `methodswith(type)` - Find methods using a specific type - `names(module)` - List names in a module - `apropos(keyword)` - Search documentation by keyword - `typeinfo(type)` - Get type information - `varinfo()` - List all variables with types and sizes

Important distinction - AI agent LSP vs NeuroREPL LSP:

Some AI agents (e.g., Kiro CLI) have their own LSP integration for analyzing project files. This is **separate** from NeuroREPL's limitation:

- **Agent LSP (e.g., Kiro with Julia LSP):**
  - Provides static code analysis for `.jl` files in your project
  - Can complete standard library functions (e.g., `println`, `map`, `filter`)
  - Can find definitions in Julia Base and installed packages
  - Works on files, not live REPL state
  - **Does NOT solve this limitation**
- **NeuroREPL LSP (not implemented):**
  - Would provide dynamic analysis of worker REPL session
  - Would complete variables defined at runtime (e.g., `x = 10` → suggest `x`)
  - Would find references to functions defined in `execute_julia()` calls
  - Would know about current workspace state and types
  - **This is what's missing**

Example of the limitation:

```
# Agent executes in NeuroREPL worker:
execute_julia(code: "my_custom_function(x) = x^2 + 1")
execute_julia(code: "data = [1, 2, 3, 4, 5]")
execute_julia(code: "result = map(my_custom_function, data)")

# Agent with Julia LSP can suggest:
# ✓ println, map, filter (standard library)
# ✓ DataFrame, CSV (if analyzing project files with those imports)

# Agent with Julia LSP CANNOT suggest:
# ✗ my_custom_function (defined in worker session)
# ✗ data (variable in worker workspace)
# ✗ result (runtime variable)
```

To fully solve this limitation:

NeuroREPL would need to integrate LanguageServer.jl into the worker process and expose LSP methods as MCP tools: - `get_completions(code, cursor_position)` - Get completion suggestions - `goto_definition(symbol)` - Find where a symbol is defined - `find_references(symbol)` - Find all uses of a symbol - `get_hover(symbol)` - Get documentation for a symbol - `get_signature_help(function, cursor)` - Get function signature

This would provide LSP features that are aware of the live REPL session state, not just static file analysis.

2. No Debugger Integration

NeuroREPL does not provide interactive debugging capabilities for code executed in the worker.

Missing capabilities: - Cannot set breakpoints in code - Cannot step through execution line-by-line - Cannot inspect variables at specific execution points - Cannot pause execution and resume interactively - No call stack inspection during execution

Current workarounds: - `@show variable` - Print variable value with name - `println("Debug: x = $x")` - Manual print debugging - `varinfo()` - Inspect all variables after execution - `@time expr` - Measure execution time - `@code_warntype expr` - Check for type instabilities - Try-catch blocks with detailed error messages

Can IDE debuggers (JetBrains, VS Code) be used?

No, IDE debuggers cannot be directly used with NeuroREPL because: - They require GUI interaction (setting breakpoints, step buttons) - They're designed for file-based debugging, not REPL string execution - They expect synchronous debugging sessions, not async MCP requests

However, the **underlying technology can be adapted**:

How Julia debuggers work: - **Debugger.jl** or **JuliaInterpreter.jl** - Core debugging engine - **Debug Adapter Protocol (DAP)** - Communication between IDE and debugger - IDE UI - Visual breakpoint management and controls

How to solve this for NeuroREPL:

Integrate Debugger.jl or JuliaInterpreter.jl into the worker and expose debugging as MCP tools:

```
# Full debugger integration (complex):
- set_breakpoint(file, line) - Set a breakpoint
- remove_breakpoint(file, line) - Remove a breakpoint
- step_into() - Step into function call
- step_over() - Step over current line
- step_out() - Step out of current function
- continue_execution() - Continue until next breakpoint
- get_stack_trace() - Get current call stack
- get_local_variables() - Get variables in current scope
- evaluate_in_context(expr) - Evaluate expression at breakpoint
```

Challenges with full debugger integration:

1. **Asynchronous nature** - Debugging is interactive (hit breakpoint pause inspect continue), but MCP is request/response
2. **State management** - Need to maintain debugger state across multiple tool calls
3. **File-based breakpoints** - Most NeuroREPL code is executed as strings via `execute_julia()`, not files
4. **Agent workflow** - AI agents would need to understand multi-step debugging (not just "run code, get result")
5. **Session complexity** - Paused debugger state conflicts with other tool calls

More practical solution - Debugging helper tools:

Instead of full interactive debugging, add tracing and inspection tools:

```
# Realistic debugging tools for NeuroREPL:
- trace_execution(code) - Execute with line-by-line output
- profile_with_trace(code) - Show execution path and timing
- inspect_at_line(code, line_number) - Show variable state at specific line
- conditional_trace(code, condition) - Trace when condition is true
- step_by_step(code) - Return execution trace with all intermediate values
```

Example of what's possible:

```julia
# Current (manual debugging):
execute_julia(code: """
    x = 10
    @show x
    y = x^2
    @show y
    result = y + 1
    @show result
""")

# Potential with debugging tools:
trace_execution(code: """
    x = 10
    y = x^2
    result = y + 1
""")
# Returns:
# Line 1: x = 10
# Line 2: y = 100 (x = 10)
# Line 3: result = 101 (y = 100)
```

Implementation path:

1. Add JuliaInterpreter.jl to worker environment
2. Create wrapper functions for tracing/inspection
3. Expose as MCP tools
4. Document debugging workflow for agents

3. Single Worker Process for MCP Requests

NeuroREPL uses a single Malt worker to handle all MCP requests, which means:

Current behavior: - All `execute_julia()` calls are serialized to one worker - Long-running code blocks other MCP requests - No concurrent execution of multiple tool calls - One agent session = one worker process

However, parallel computation IS possible:

Agents can use **Distributed.jl inside the worker** for parallel computation:

```julia
execute_julia(code: """
    using Distributed
    addprocs(4)  # Add 4 worker processes

    @everywhere function work(x)
        return x^2
    end

    result = pmap(work, 1:1000)  # Parallel execution
""")
```

This provides: - □ Parallel computation within a single MCP session - □ Standard Julia parallel programming (`pmap`, `@distributed`, `@everywhere`) - □ Shared state across parallel tasks - □ Does NOT enable concurrent MCP requests (still serialized)

Potential solutions for concurrent MCP requests:

1. **Worker pool** - NeuroREPL maintains multiple Malt workers
   - Pros: Concurrent MCP requests, better resource utilization
   - Cons: Workers don't share state, complex session management
2. **Session-based workers** - One Malt worker per MCP session
   - Pros: Session isolation, concurrent sessions
   - Cons: Higher memory usage, session affinity required
3. **Hybrid approach** - Single worker + Distributed.jl (current + user-managed)

- Pros: No NeuroREPL changes needed, full user control
- Cons: Agent must manage parallelism, doesn't help with concurrent MCP requests

See "Parallel Processing Workflow" in Usage Patterns for examples.

4. No Security Layer

- No authentication or authorization
- Code execution is unrestricted
- NOTE: This is intentional, assumption is that the Agents and NeuroREPL operate in a truested network environment

5. Limited Error Recovery

NeuroREPL does not automatically recover from worker failures or provide retry logic.

Current behavior: - Worker crash (segfault, OOM, unrecoverable error) requires manual `restart_session()` - No automatic detection of dead workers - No retry on transient failures - No timeout protection for long-running code - No workspace state preservation

Manual recovery: "'julia # Check worker status worker_status() # Returns: "Worker: Dead (not running)"

Restart manually restart_session() # Spawns new worker, clears all state "'

How to solve this:

1. Automatic worker health monitoring:

```
# Monitor worker and auto-restart if dead
function monitor_worker_health()
    @async while SERVER[] !== nothing
        sleep(5)  # Check every 5 seconds
        if WORKER[] !== nothing && !Malt.isrunning(WORKER[])
            @warn "Worker died, auto-restarting..."
            restart_worker!()
        end
    end
end
```

2. Automatic retry on worker failure:

```
# Retry execution if worker crashes
function execute_code_with_retry(args, max_retries=3)
    for attempt in 1:max_retries
        try
            return execute_code(args)
        catch e
            if WORKER[] === nothing || !Malt.isrunning(WORKER[])
                @warn "Worker crashed on attempt $attempt/$max_retries"
                restart_worker!()
                if attempt == max_retries
                    return "Error: Worker crashed after $max_retries attempts: $e
                end
            else
                rethrow(e)  # Not a worker crash, propagate error
            end
        end
    end
end
```

3. Timeout protection:

```
# Prevent infinite loops from blocking forever
function execute_code_with_timeout(args, timeout_seconds=300)
```

```
    start_time = time()
    task = @async execute_code(args)

    while !istaskdone(task)
        if time() - start_time > timeout_seconds
            @warn "Execution timeout, interrupting worker..."
            Malt.interrupt(WORKER[])
            sleep(1)
            if !Malt.isrunning(WORKER[])
                restart_worker!()
            end
            return "Error: Execution timeout after $timeout_seconds secor
        end
        sleep(0.1)
    end

    return fetch(task)
end
```

4. Graceful degradation:

```
# Return informative errors instead of crashing
function safe_execute(args)
    try
        if WORKER[] === nothing || !Malt.isrunning(WORKER[])
            @info "Worker not running, initializing..."
            init_worker!()
        end
        return execute_code(args)
    catch e
        return Dict(
            "error" => true,
            "message" => sprint(showerror, e),
            "worker_status" => WORKER[] !== nothing && Malt.isrunning(WOR
            "suggestion" => "Try restart_session() to recover",
            "backtrace" => sprint(showerror, e, catch_backtrace())
        )
    end
end
```

5. Workspace state preservation:

```
# Periodically save workspace state
function save_workspace_snapshot()
    execute_julia(code: """
        using Serialization
        workspace_state = Dict(
            "variables" => String.(names(Main, all=false, imported=false)),
            "timestamp" => time()
        )
        serialize("workspace_backup.jls", workspace_state)
        println("Workspace snapshot saved")
    """)
end

# Restore after crash (limited - can't restore all types)
function restore_workspace_info()
    execute_julia(code: """
        using Serialization
        if isfile("workspace_backup.jls")
            backup = deserialize("workspace_backup.jls")
            println("Previous workspace had variables: ", backup["variables"])
            println("Snapshot from: ", backup["timestamp"])
        else
            println("No workspace backup found")
        end
    """)
end
```

Implementation priorities:

1. **Health monitoring** (easy) - Background task to detect dead workers
2. **Graceful degradation** (easy) - Better error messages and auto-init
3. **Timeout protection** (medium) - Prevent infinite loops
4. **Automatic retry** (medium) - Retry on transient failures
5. **State preservation** (hard) - Limited by Julia serialization constraints

Current workarounds:

- Use `worker_status()` to check health before critical operations
- Use `interrupt_worker!()` to stop long-running code
- Use `restart_session()` to recover from crashes
- Keep important code in files (use `include()`) so it can be re-executed after restart

6. No Binary/Image Output Support

NeuroREPL only returns text responses - binary data like plots, images, or other media cannot be transmitted to agents.

Current behavior:

All tool responses are converted to strings via `string(result)`:

```
# In handle_jsonrpc:
result = tool.handler(arguments)
return Dict(
    "result" => Dict(
        "content" => [Dict("type" => "text", "text" => string(result))],
        "isError" => false
    )
)
```

What happens with plots:

```
# Agent executes:
execute_julia(code: """
    using Plots
    p = plot(1:10, rand(10))
""")

# Returns:
# "Plot{Plots.GRBackend() n=1}"  # Just object description, not the image
```

The agent cannot see the actual plot image.

Affected use cases:

- Plots (Plots.jl, Makie.jl, Gadfly.jl)
- Images (Images.jl, ImageView.jl)
- DataFrames with rich display
- LaTeX equations
- HTML output
- Any binary data

How to solve this:

1. Detect and encode binary content:

```
function format_result(result)
    # Handle Plots.jl
    if isdefined(Main, :Plots) && result isa Main.Plots.Plot
        temp = tempname() * ".png"
        Main.Plots.savefig(result, temp)
        data = base64encode(read(temp))
        rm(temp)
        return [Dict(
            "type" => "image",
            "data" => data,
            "mimeType" => "image/png"
```

```
        )]
    end

    # Handle Images.jl
    if isdefined(Main, :Images) && result isa AbstractArray
        if eltype(result) <: Main.Images.Colorant
            # Convert to PNG and encode
            temp = tempname() * ".png"
            Main.Images.save(temp, result)
            data = base64encode(read(temp))
            rm(temp)
            return [Dict(
                "type" => "image",
                "data" => data,
                "mimeType" => "image/png"
            )]
        end
    end

    # Default: text
    return [Dict("type" => "text", "text" => string(result))]
end
```

2. Use MCP content types:

MCP protocol supports multiple content types: - `text` - Plain text (current) - `image` - Base64-encoded image with mimeType - `resource` - Reference to external resource

3. Implement in tool handler:

```
# Modified execute_code function:
function execute_code(args)
    # ... existing code ...
    result = Malt.remote_eval_fetch(WORKER[], ...)

    # Format based on type
    content = format_result(result)

    return Dict(
        "content" => content,  # Can be text, image, or mixed
        "isError" => false
    )
end
```

4. Support multiple outputs:

```
# Return both text and image
execute_julia(code: """
    using Plots
    p = plot(1:10, rand(10))
    println("Generated plot with 10 points")
    p
""")

# Could return:
# [
#   {"type": "text", "text": "Generated plot with 10 points"},
#   {"type": "image", "data": "base64...", "mimeType": "image/png"}
# ]
```

Current workaround:

```
# Save to file and tell agent
execute_julia(code: """
    using Plots
    p = plot(1:10, rand(10))
    savefig(p, "plot.png")
```

```
    "Plot saved to plot.png"
""")

# Agent would need file system access to read plot.png
# (Not ideal, but works if agent has fs_read tool)
```

Implementation priority: Medium-High

This would significantly improve agent capabilities for: - Data visualization - Image processing - Scientific computing - Exploratory data analysis

Note: Some AI agents (like Kiro CLI) can display images if provided in base64 format, making this enhancement immediately useful.

7. No Explicit Session Management in MCP Protocol

NeuroREPL uses implicit session management - the server manages worker lifecycle outside the MCP protocol.

Current NeuroREPL architecture: - **Synchronous** - One request at a time, blocking - **Long-lived session** - Worker starts once, persists until restart - **Session management** - Handled outside MCP protocol (server manages worker lifecycle) - **Single session** - One implicit session per connection - **Simple for agents** - Just call tools, session persists automatically

Alternative approach: Session lifecycle in MCP protocol

Some MCP servers make session management explicit as part of the protocol:

```
# Session management as MCP methods:
- "session/create" - Start a new Julia session
- "session/destroy" - Stop a Julia session
- "session/list" - List active sessions
- "session/switch" - Switch between sessions
```

This makes session lifecycle **explicit in the protocol** rather than implicit server behavior.

Proposed architecture with explicit sessions: - **Asynchronous** - Multiple concurrent requests - **Session lifecycle in MCP** - Starting/stopping sessions is part of the protocol itself - **Ephemeral or managed sessions** - Sessions can be created/destroyed via MCP messages - **Multiple sessions** - Agents explicitly create/destroy sessions - **Flexible but complex** - Can have multiple sessions per connection

Comparison:

Aspect	Current (Implicit)	Proposed (Explicit)
Session control	Automatic	Agent-controlled
Multiple sessions	No	Yes
Concurrency	Serialized	Concurrent
Complexity	Simple	More complex
Agent responsibility	None	Manage lifecycle
Use case	Single-agent workflows	Multi-session scenarios

Pros of explicit session management: - Agents have explicit control over sessions - Can run multiple isolated sessions concurrently - Clear session boundaries - Better for multi-tenant scenarios - Supports concurrent execution of different tasks

Cons of explicit session management: - More complex for simple use cases - Agents must manage session lifecycle - More protocol overhead - Requires agents to understand session concepts

Is this "the way forward"?

It depends on use case: - **Simple agent workflows** - Current implicit approach is fine - **Multi-user/multi-session** - Explicit session-in-protocol is better - **Concurrent execution** - Async + session management is better - **Single agent, single task** - Current approach is simpler

Implementation considerations:

If NeuroREPL were to add explicit session management:

1. Add session MCP methods (`session/create`, `session/destroy`, etc.)
2. Maintain worker pool (one worker per session)
3. Route requests to correct worker based on session ID
4. Handle session cleanup and resource management
5. Support concurrent requests across sessions

Conclusion: Both approaches are valid. NeuroREPL's current implicit approach is simpler for single-agent scenarios. Explicit session management would be better for complex multi-session scenarios but adds significant complexity.

8. No Full REPL Features in Worker Mode

The current `:w` worker mode provides code execution but lacks full Julia REPL features like package mode (`]`), shell mode (`;`), and help mode (`?`).

Current `:w` mode capabilities: - □ Execute Julia code - □ Access variables and functions - □ Load packages via `using Pkg; Pkg.add("Package")` - □ Run shell commands via `run(\`command`)- □ Get help via[**doc?**] function_name- □ Package mode (]) - Nost,add Package,rm Packageshortcuts - □ Shell mode (;) - No direct shell command execution - □ Help mode (?) - No?function` syntax - □ Tab completion for context-aware suggestions

Why these don't work:

The `:w` mode uses `Malt.remote_eval_fetch()` to send code strings to the worker. Special REPL modes (`]`, `;`, `?`) are **REPL parser features**, not Julia expressions. They're handled by the REPL's input parser before code execution.

Attempted solutions and their limitations:

1. RemoteREPL.jl: - Provides remote connection to worker REPL - Supports basic code execution - □ Does NOT support special REPL modes (`]`, `;`, `?`) - Essentially same as current `:w` mode with nicer interface

2. Distributed.jl instead of Malt.jl: - Same limitation - workers are for remote code execution, not interactive REPLs - Would not solve the problem

3. Separate terminal with `nc`: - Could connect to RemoteREPL server in worker - Still no special REPL modes - Requires separate terminal window

Working solution: ReplMaker.jl

ReplMaker.jl can create custom REPL modes that **mimic** the special modes:

```julia
using ReplMaker

# Create custom package mode for worker
function parse_pkg_command(code)
    # Convert Pkg REPL shortcuts to function calls
    code = replace(code, r"^st\b" => "status()")
    code = replace(code, r"^add\s+(.+)" => s"add(\"\1\")")
    code = replace(code, r"^rm\s+(.+)" => s"rm(\"\1\")")
    code = replace(code, r"^up\b" => "update()")

    return Malt.remote_eval_fetch(
        NeuroREPL.WORKER[],
        Meta.parse("using Pkg; Pkg.$code")
    )
end

initrepl(
    parse_pkg_command,
    prompt_text = "pkg> ",

    prompt_color = :blue,
    start_key = ']',
    mode_name = "Worker Package Mode"
)
```

This approach: - □ Creates custom `]` mode that works with worker - □ Parses Pkg shortcuts (`st`, `add`, `rm`) into function calls - □ Only requires ReplMaker on server side (not in worker) - □ Can create similar modes for shell (`;`) and help (`?`) - □ Requires implementing parsing for each shortcut - □ Not identical to real REPL modes (some features may be missing)

Implementation requirements:

1. **Package mode parser** (~50 lines):
    - `st` □ `status()`
    - `add Package` □ `add("Package")`
    - `rm Package` □ `rm("Package")`
    - `up` □ `update()`
    - `build Package` □ `build("Package")`
2. **Shell mode parser** (~20 lines):
    - Parse command and wrap in `run(\…‘)‘
    - Handle output capture
3. **Help mode parser** (~30 lines):
    - `?function` □ `@doc function`
    - Handle module-qualified names
4. **Integration into `:w` mode**:
    - Replace current `:w` implementation with ReplMaker-based mode
    - Register sub-modes for `]`, `;`, `?`
    - Maintain backward compatibility

Example of working implementation:

```julia
# Tested and working:
julia> initrepl(
    code -> Malt.remote_eval_fetch(
        NeuroREPL.WORKER[],
        Meta.parse("using Pkg; Pkg.$code")
    ),
    prompt_text = "wpkg> ",
    prompt_color = :blue,
    start_key = '}',
    mode_name = "Worker Package Mode"
)

wpkg> status()  # Works!
# Shows worker environment packages
```

Limitations of ReplMaker approach:

- Not 100% compatible with real Pkg REPL mode
- Some advanced features may not work
- Requires maintaining parser code
- Tab completion limited to Julia syntax (not context-aware)

Alternative: Accept current limitations

The current `:w` mode is actually quite functional: - Can do everything via function calls - `using Pkg; Pkg.add("Package")` instead of `]add Package` - `run(`\ls‘)`instead of`;ls-[**doc?**] println`instead of`?println‘

Recommendation:

Implement ReplMaker-based modes for better UX, but document that they’re approximations of real REPL modes. The ~100 lines of parsing code is worth it for significantly improved user experience.

Implementation priority: Medium

This would make the worker mode feel much more like a real Julia REPL while maintaining the Malt.jl architecture benefits.

9. Multi-Model Solution Validation and Ranking:

Transform NeuroREPL from a simple execution environment into a **solution validation and ranking system** for multi-model AI workflows. This would enable agents to test multiple solution candidates and automatically select the best one based on correctness, performance, and other metrics.

Use cases:

1. **Consensus-based problem solving** - Multiple AI models propose solutions, NeuroREPL validates and ranks them
2. **Automated optimization** - Test different implementations and choose the fastest/most efficient
3. **Correctness verification** - Run multiple solutions against test cases and select the correct one
4. **Performance benchmarking** - Compare solutions from different models and pick the winner

Proposed MCP tools:

```
# 1. Execute with validation
execute_with_validation(
    code: "function f(x) x^2 end",
    test_code: "@test f(2) == 4; @test f(-3) == 9",
    metrics: ["correctness", "performance"]
)
# Returns: {success: true, time: "0.001ms", allocations: 0, tests_passed: 2}

# 2. Benchmark multiple solutions
benchmark_solutions(
    solutions: [
        {name: "claude_solution", code: "sum(x.^2)"},
        {name: "gpt4_solution", code: "mapreduce(x->x^2, +, x)"},
        {name: "gemini_solution", code: "x' * x"}
    ],
    test_input: "x = rand(10000)",
    iterations: 1000
)
# Returns: {
#   winner: "gemini_solution",
#   rankings: [{name: "gemini_solution", time: "0.001ms", allocations: 0}, ...],
#   speedup: "10x faster than slowest"
# }

# 3. Test multiple solutions with validation
test_multiple_solutions(
    solutions: [
        {model: "claude", code: "function opt1(x) ... end"},
```

```
        {model: "gpt4", code: "function opt2(x) ... end"},
        {model: "gemini", code: "function opt3(x) ... end"}
    ],
    test_cases: [
        {input: "[1,2,3]", expected: "14"},
        {input: "[0]", expected: "0"},
        {input: "[]", expected: "0"}
    ],
    benchmark: true
)
# Returns: {
#   best: {model: "gemini", code: "...", score: 100},
#   results: [
#     {model: "claude", correctness: 100%, time: "0.5ms", score: 85},
#     {model: "gpt4", correctness: 100%, time: "0.3ms", score: 90},
#     {model: "gemini", correctness: 100%, time: "0.1ms", score: 100}
#   ]
# }

# 4. Consensus execution
consensus_execute(
    code_variants: [
        "result = sum(x.^2)",
        "result = mapreduce(x->x^2, +, x)",
        "result = dot(x, x)"
    ],
    input: "x = [1, 2, 3, 4, 5]",
    require_consensus: true
)
# Returns: {
#   consensus: true,
#   value: "55",
#   agreement: "3/3 solutions agree",
#   fastest: "dot(x, x)"
# }

# 5. Iterative solution improvement
improve_solution(
    initial_code: "function slow_sum(x) sum([i^2 for i in x]) end",
    optimization_goal: "performance",
    max_iterations: 5,
    test_cases: [{input: "1:10000", expected: "333383335000"}]
)
# Returns: {
#   iterations: 3,
#   improvements: [
#     {iteration: 1, code: "...", speedup: "2x"},
#     {iteration: 2, code: "...", speedup: "5x"},
#     {iteration: 3, code: "...", speedup: "10x"}
#   ],
#   final_solution: "function fast_sum(x) sum(x.^2) end",
#   total_speedup: "10x"
# }
```

Implementation architecture:

```
# Solution validation framework
struct SolutionCandidate
    model::String           # Which AI model proposed it
    code::String           # The solution code
    metadata::Dict         # Additional info (timestamp, model version, etc.)
end

struct ValidationResult
    candidate::SolutionCandidate
    correctness::Float64   # 0.0-1.0 (percentage of tests passed)
    performance::Float64   # Execution time in ms
```

```
    allocations::Int       # Memory allocations
    score::Float64         # Combined metric
    errors::Vector{String} # Any errors encountered
end

# Ranking strategies
function rank_solutions(results::Vector{ValidationResult}, strategy::Symb
    if strategy == :correctness_first
        # Prioritize correctness, then performance
        sort(results, by = r -> (-r.correctness, r.performance))
    elseif strategy == :performance_first
        # Only consider correct solutions, rank by speed
        correct = filter(r -> r.correctness == 1.0, results)
        sort(correct, by = r -> r.performance)
    elseif strategy == :balanced
        # Weighted score: 70% correctness, 30% performance
        sort(results, by = r -> -(0.7 * r.correctness + 0.3 * (1.0 / r.pe
    end
end
```

Example workflow:

```
# Agent orchestrator asks 3 models to solve a problem
problem = "Optimize this function: sum_of_squares(x) = sum([i^2 for i in

# Model 1 (Claude): Suggests list comprehension optimization
solution1 = "sum_of_squares(x) = sum(i^2 for i in x)"

# Model 2 (GPT-4): Suggests vectorization
solution2 = "sum_of_squares(x) = sum(x.^2)"

# Model 3 (Gemini): Suggests dot product
solution3 = "sum_of_squares(x) = dot(x, x)"
```

```
# Agent calls NeuroREPL to validate and rank
result = test_multiple_solutions(
    solutions: [
        {model: "claude", code: solution1},
        {model: "gpt4", code: solution2},
        {model: "gemini", code: solution3}
    ],
    test_cases: [
        {input: "[1,2,3]", expected: "14"},
        {input: "1:10000", expected: "333383335000"}
    ],
    benchmark: true
)

# NeuroREPL returns:
# {
#   best: {model: "gemini", code: "dot(x, x)", score: 100},
#   winner_reason: "Fastest (0.001ms) with 100% correctness",
#   rankings: [
#     {model: "gemini", correctness: 100%, time: "0.001ms", score: 100},
#     {model: "gpt4", correctness: 100%, time: "0.010ms", score: 90},
#     {model: "claude", correctness: 100%, time: "0.050ms", score: 75}
#   ]
# }

# Agent automatically uses the best solution
execute_julia(code: "sum_of_squares(x) = dot(x, x)")
```

Benefits:

1. **Automated quality assurance** - No need for human to evaluate multiple solutions
2. **Performance optimization** - Automatically find the fastest correct solution

3. **Consensus validation** - Verify correctness when multiple models agree
4. **Iterative improvement** - Test variations and improvements automatically
5. **Model comparison** - Track which models produce better solutions for different problems
6. **Reduced hallucination risk** - Cross-validate solutions from multiple sources

Advanced features:

- **Type stability checking** - Rank solutions by type inference quality
- **Memory efficiency** - Consider allocations in ranking
- **Compilation time** - Factor in first-run compilation overhead
- **Readability scoring** - Use code complexity metrics
- **Test coverage** - Generate additional test cases automatically
- **Regression testing** - Ensure new solutions don't break existing functionality

Integration with existing tools:

```
# Combine with existing NeuroREPL tools
test_multiple_solutions(
    solutions: [...],
    validation_tools: [
        "code_warntype",  # Check type stability
        "profile",        # Profile performance
        "time"           # Measure execution time
    ]
)
```

Implementation priority: Medium

10. Parallel Agent Collaboration (Anthropic-style):

Enable multiple AI agent instances to collaborate on a single large project by working in parallel on different parts of the codebase. Inspired by Anthropic's parallel Claude approach for building complex software systems.

Concept:

Instead of one agent working sequentially, spawn multiple agent instances that can work simultaneously on different aspects of a project, coordinating through a shared codebase and task management system.

Architecture:

```
NeuroREPL Server
    │
    ├─ Worker Pool (multiple Malt workers)
    │   ├─ Worker 1: Agent A (working on parser)
    │   ├─ Worker 2: Agent B (working on optimizer)
    │   ├─ Worker 3: Agent C (working on tests)
    │   └─ Worker 4: Agent D (maintaining docs)
    │
    └─ Coordination Layer
        ├─ Task queue and locking
        ├─ Git-based synchronization
        └─ Progress tracking
```

Proposed MCP tools:

```
# 1. Claim a task (with locking)
claim_task(
    task_id: "implement_parser",
    agent_id: "agent_a",
    estimated_time: "30min"
)
# Returns: {locked: true, task_file: "current_tasks/implement_parser.txt"

# 2. Report progress
report_progress(
    task_id: "implement_parser",
    status: "in_progress",
    completion: 60,
    notes: "Basic parsing working, need to handle edge cases"
)

# 3. Complete task and release lock
complete_task(
    task_id: "implement_parser",
    result: "success",
    next_tasks: ["implement_ast_validation", "add_parser_tests"]
)

# 4. Query available tasks
list_available_tasks(
    filter: "priority:high",
    exclude_locked: true
)
# Returns: [
#   {id: "optimize_codegen", priority: "high", estimated: "1h"},
#   {id: "fix_memory_leak", priority: "high", estimated: "30min"}
# ]

# 5. Synchronize with other agents
sync_workspace(
    pull_changes: true,
    resolve_conflicts: "auto",
    push_changes: true
)
# Returns: {
#   conflicts: 2,
#   resolved: 2,
```

```
#   files_updated: 15,
#   status: "synced"
# }

# 6. View other agents' status
agent_status(
    show_all: true
)
# Returns: [
#   {agent: "agent_a", task: "implement_parser", progress: 60%},
#   {agent: "agent_b", task: "optimize_codegen", progress: 30%},
#   {agent: "agent_c", task: "idle", waiting_for: "agent_a"}
# ]
```

Task coordination system:

```
# Task management structure
struct Task
    id::String
    description::String
    priority::Symbol  # :critical, :high, :medium, :low
    dependencies::Vector{String}  # Task IDs that must complete first
    estimated_time::Int  # Minutes
    assigned_to::Union{String,Nothing}  # Agent ID or nothing
    status::Symbol  # :available, :locked, :in_progress, :completed
    created_at::Float64
    locked_at::Union{Float64,Nothing}
end

# Git-based locking (similar to Anthropic's approach)
function claim_task(task_id::String, agent_id::String)
    # Try to create lock file
    lock_file = "current_tasks/$(task_id).txt"

    # Write agent ID and timestamp
    write(lock_file, """
        Agent: $agent_id
        Started: $(now())
        Task: $task_id
    """)

    # Try to commit and push
    try
        run(`git add $lock_file`)
        run(`git commit -m "Agent $agent_id claiming $task_id"`)
        run(`git push`)
        return (success=true, locked=true)
    catch e
        # Another agent claimed it first
        run(`git reset --hard HEAD`)
        run(`git pull`)
        return (success=false, locked=false, reason="Already claimed")
    end
end
```

Agent specialization roles:

```
# Different agents can have different roles

# Implementation agent
agent_role(
    role: "implementer",
    focus: "core_functionality",
    skills: ["algorithm_design", "optimization"]
)

# Testing agent

agent_role(
    role: "tester",
    focus: "quality_assurance",
    skills: ["test_design", "edge_case_discovery"]
)

# Documentation agent
agent_role(
    role: "documenter",
    focus: "documentation",
    skills: ["technical_writing", "code_explanation"]
)

# Code quality agent
agent_role(
    role: "reviewer",
    focus: "code_quality",
    skills: ["refactoring", "performance_optimization"]
)
```

Example workflow:

```
# Project: Build a Julia package for data analysis

# Agent A (Implementer):
claim_task(task_id: "implement_data_loader")
execute_julia(code: """
    function load_data(path::String)
        # Implementation
    end
""")
complete_task(task_id: "implement_data_loader")

# Agent B (Tester) - works in parallel:
claim_task(task_id: "write_loader_tests")
execute_julia(code: """
    @testset "Data Loader" begin
        @test load_data("test.csv") isa DataFrame
    end
""")
complete_task(task_id: "write_loader_tests")

# Agent C (Documenter) - works in parallel:
claim_task(task_id: "document_api")
execute_julia(code: """
    \"\"\"
        load_data(path::String) -> DataFrame

    Load data from CSV file at `path`.
    \"\"\"
""")
complete_task(task_id: "document_api")

# All agents sync periodically
sync_workspace(pull_changes: true, push_changes: true)
```

Key features:

1. **Git-based synchronization** - Agents coordinate through git commits/pushes
2. **Task locking** - Prevents duplicate work via file-based locks
3. **Automatic conflict resolution** - Agents handle merge conflicts
4. **Progress tracking** - Shared progress files for coordination
5. **Role specialization** - Different agents focus on different aspects
6. **Parallel execution** - Multiple agents work simultaneously

Challenges:

- **Merge conflicts** - Frequent when multiple agents modify same files
- **Task dependencies** - Need to track what can be done in parallel
- **Communication overhead** - Agents need to understand each other's work
- **Quality control** - No single agent has full context
- **Resource management** - Multiple workers consume more memory

Benefits:

- **Faster development** - Parallel work on independent tasks
- **Specialization** - Agents can focus on their strengths
- **Scalability** - Add more agents for larger projects
- **Continuous progress** - Always something being worked on

Implementation priority: High

This would enable NeuroREPL to tackle large, complex projects that are beyond the scope of a single agent session.

11. Meta-AI System: Competitive + Collaborative:

Combine multi-model validation (competitive) with parallel agent collaboration (collaborative) to create a comprehensive AI development system that leverages both competition between models and collaboration between agents.

Concept:

Multiple AI models compete to propose the best high-level solution, then for each solution, multiple agent instances collaborate to implement it in parallel. NeuroREPL validates and ranks all implementations to select the ultimate winner.

Architecture:

```
Phase 1: Competitive (Multi-Model)
    │
    ├─ Claude proposes Solution A
    ├─ GPT-4 proposes Solution B
    └─ Gemini proposes Solution C
    │
    ▼
Phase 2: Collaborative (Parallel Agents per Solution)
    │
    ├─ Solution A Implementation
    │   ├─ Agent A1: Core logic
    │   ├─ Agent A2: Tests
    │   └─ Agent A3: Optimization
    │
    ├─ Solution B Implementation
    │   ├─ Agent B1: Core logic
    │   ├─ Agent B2: Tests
    │   └─ Agent B3: Optimization
    │
    └─ Solution C Implementation
        ├─ Agent C1: Core logic
        ├─ Agent C2: Tests
        └─ Agent C3: Optimization
    │
    ▼
Phase 3: Validation & Ranking
```

```
│
└─ NeuroREPL tests all implementations
    └─ Selects best overall solution
```

Proposed MCP tools:

```
# 1. Orchestrate competitive-collaborative workflow
meta_solve(
    problem: "Optimize matrix multiplication for large sparse matrices",
    models: ["claude-opus", "gpt-4", "gemini-pro"],
    agents_per_solution: 3,
    validation_criteria: {
        correctness: 1.0,
        performance: 0.8,
        memory: 0.6,
        code_quality: 0.4
    },
    timeout: "2 hours"
)

# 2. Track multi-model competition
competition_status()
# Returns: {
#   phase: "implementation",
#   solutions: [
#     {
#       model: "claude",
#       approach: "Block-based sparse multiplication",
#       agents: [
#         {id: "claude_a1", task: "core_impl", progress: 80%},
#         {id: "claude_a2", task: "tests", progress: 60%},
#         {id: "claude_a3", task: "optimize", progress: 40%}
#       ]
#     },
#     {model: "gpt4", approach: "CSR format optimization", agents: [...]},
#     {model: "gemini", approach: "Parallel COO algorithm", agents: [...]}
#   ]
# }

# 3. Compare implementations in real-time
live_comparison(
    metric: "performance",
    test_input: "sparse_matrix_10000x10000"
)
# Returns: {
#   current_leader: "gemini",
#   rankings: [
#     {model: "gemini", time: "0.5s", status: "complete"},
#     {model: "claude", time: "0.8s", status: "complete"},
#     {model: "gpt4", time: "N/A", status: "in_progress"}
#   ]
# }

# 4. Final validation and selection
validate_and_select(
    require_all_complete: false,
    early_stopping: true,
    confidence_threshold: 0.95
)
# Returns: {
#   winner: {
#     model: "gemini",
#     approach: "Parallel COO algorithm",
#     score: 98.5,
#     metrics: {
#       correctness: 100%,
#       performance: "0.5s (2x faster)",
```

```
#       memory: "50MB (30% less)",
#       code_quality: 9.2/10
#     }
#   },
#   runner_up: {model: "claude", score: 92.3},
#   all_results: [...]
# }
```

Detailed workflow:

```
# Step 1: Problem definition
problem = """
Implement an efficient sparse matrix multiplication algorithm
that can handle matrices up to 100,000 x 100,000 with <1% density.
Must be faster than naive implementation and use less memory.
"""

# Step 2: Multi-model proposal phase (competitive)
proposals = request_proposals(
    problem: problem,
    models: ["claude-opus-4.6", "gpt-4-turbo", "gemini-2.0-pro"],
    require: ["algorithm_description", "complexity_analysis", "implementa
)

# Returns:
# [
#   {
#     model: "claude",
#     approach: "Block-based CSR with cache optimization",
#     complexity: "O(nnz * log(n))",
#     plan: ["Implement CSR format", "Add blocking", "Optimize cache"]
#   },
#   {
#     model: "gpt4",
#     approach: "Parallel COO with work stealing",
#     complexity: "O(nnz) with parallelism",
#     plan: ["COO format", "Thread pool", "Work stealing scheduler"]
#   },
#   {
#     model: "gemini",
#     approach: "Hybrid CSR/COO with SIMD",
#     complexity: "O(nnz) with SIMD speedup",
#     plan: ["Hybrid format", "SIMD operations", "Auto-tuning"]
#   }
# ]

# Step 3: Parallel implementation phase (collaborative per solution)
implementations = parallel_implement(
    proposals: proposals,
    agents_per_solution: 4,
    agent_roles: ["implementer", "tester", "optimizer", "documenter"],
    time_limit: "1 hour"
)

# Each solution gets 4 agents working in parallel:
# - Agent 1: Implements core algorithm
# - Agent 2: Writes comprehensive tests
# - Agent 3: Optimizes performance
# - Agent 4: Writes documentation

# Step 4: Continuous validation during implementation
while any_in_progress(implementations)
    # Test completed implementations
    for impl in filter(is_complete, implementations)
        result = validate_implementation(
            code: impl.code,
            tests: impl.tests,
            benchmark: true
```

```
        )

        # Early stopping if one solution is clearly superior
        if result.score > 95 && result.correctness == 100%
            stop_other_implementations()
            return result
        end
    end

    sleep(30)  # Check every 30 seconds
end

# Step 5: Final comprehensive validation
final_results = comprehensive_validation(
    implementations: implementations,
    test_suite: "extensive",
    benchmarks: [
        {name: "small_sparse", size: "1000x1000", density: 0.01},
        {name: "medium_sparse", size: "10000x10000", density: 0.005},
        {name: "large_sparse", size: "100000x100000", density: 0.001}
    ],
    metrics: [
        "correctness",
        "performance",
        "memory_usage",
        "code_quality",
        "maintainability"
    ]
)

# Step 6: Select winner and provide detailed report
winner = select_winner(
    results: final_results,
    weights: {
        correctness: 1.0,      # Must be correct
        performance: 0.8,      # Very important
        memory_usage: 0.6,     # Important
        code_quality: 0.4,     # Nice to have
        maintainability: 0.3   # Nice to have
    }
)

# Returns comprehensive report:
# {
#   winner: {
#     model: "gemini",
#     approach: "Hybrid CSR/COO with SIMD",
#     final_score: 97.8,
#     implementation_time: "45 minutes",
#     agents_used: 4,
#     code_lines: 450,
#     test_coverage: 98%,
#     performance: {
#       small: "0.001s (10x faster than baseline)",
#       medium: "0.05s (15x faster)",
#       large: "0.5s (20x faster)"
#     },
#     memory: {
#       small: "1MB (50% less)",
#       medium: "10MB (60% less)",
#       large: "50MB (70% less)"
#     }
#   },
#   all_results: [
#     {model: "gemini", score: 97.8, rank: 1},
#     {model: "claude", score: 94.2, rank: 2},
```

```
#     {model: "gpt4", score: 91.5, rank: 3}
#   ],
#   insights: [
#     "Gemini's SIMD approach provided significant speedup",
#     "Claude's cache optimization was effective but complex",
#     "GPT-4's parallel approach had higher memory overhead"
#   ]
# }
```

Key advantages of combined approach:

1. **Best of both worlds**
   - Competition ensures diverse approaches
   - Collaboration ensures quality implementation
2. **Redundancy and validation**
   - Multiple models reduce hallucination risk
   - Multiple agents catch implementation errors
3. **Parallel exploration**
   - Different models explore different solution spaces
   - Different agents explore different implementation strategies
4. **Automatic quality assurance**
   - Competitive pressure drives quality
   - Collaborative testing ensures correctness
5. **Scalability**
   - Add more models for more diverse solutions
   - Add more agents for faster implementation
6. **Learning and improvement**
   - Track which models excel at which problems
   - Track which agent roles are most effective

Resource requirements:

```
# Example: 3 models × 4 agents each = 12 parallel workers

resources = {
    models: 3,
    agents_per_model: 4,
    total_workers: 12,
    estimated_memory: "12 × 150MB = 1.8GB",
    estimated_time: "1-2 hours",
    estimated_cost: "$50-100 in API calls"
}
```

Use cases:

1. **Critical algorithms** - When correctness and performance are paramount
2. **Research problems** - Explore multiple solution approaches
3. **Optimization challenges** - Find the absolute best solution
4. **Complex systems** - Large projects requiring diverse expertise
5. **Competitive benchmarking** - Compare AI model capabilities

Implementation priority: Medium-High

This represents the ultimate evolution of NeuroREPL: from a simple execution environment to a sophisticated meta-AI orchestration system that combines the strengths of multiple models and multiple agents to solve complex problems with unprecedented quality and efficiency.

Future vision:

Imagine asking NeuroREPL to “build a high-performance web server in Julia” and having it: 1. Ask Claude, GPT-4, and Gemini for their architectural proposals 2. Spawn 4 agents per proposal to implement them in parallel 3. Continuously test and benchmark all implementations 4. Select the best one based on performance, correctness, and code quality 5. Deliver a production-ready solution in hours instead of weeks

This is the future of AI-assisted software development.

This would position NeuroREPL as a **meta-AI tool** - not just executing code, but intelligently evaluating and selecting the best solutions from multiple AI models. Particularly valuable for: - Complex optimization problems - Performance-critical code - Safety-critical applications requiring validation - Research comparing different AI models' coding capabilities